

Visual Augmentation of Audio Semantics for Accessibility

Technical report

Adam Gavriely

Bar-Ilan University, Department of Computer Science

MSc final project, submitted to Oren Glickman

3 October 2026

Code and data: <https://github.com/adamgavriely/MscFinalProject>, release v1.3.0

Abstract

Subtitles carry speech, but deaf and hard-of-hearing viewers still miss the other sounds of a video. Captioning practice need not describe a sound whose source is clearly seen on screen. The sounds a deaf viewer loses are therefore those whose source is *off screen*: a siren behind the camera, a dog barking in the next room. This project builds a system that finds them and shows a generated picture of each one beside the video while it is heard.

Nothing is trained: the system chains open sound detectors, audio-language “listeners”, a vision-language model that checks whether the source is visible, and a text-to-image model. Joining them is not plug and play: each model fails in a way the next one cannot see. Detectors miss quiet sounds under speech, a lone listener’s “yes” is right about one time in three, and the vision model answers “is the object there?” instead of “is it making the sound now?”. Even with the full detection stack, drawing every detected sound without reading the video shows more than twice as many wrong pictures as right ones on the test set. The final system is the result of 66 pre-registered rounds, each change accepted only by a rule written before its run, over more than 150 full-pipeline variants and more than 30 open models.

On a test set of 88 clips, scored per sound against human labels, the system finds 24 of 65 needed off-screen sounds, keeping 10 of the 13 danger sounds, with 24 wrong pictures. Its viewer cost (a miss costs 4, a wrong picture 2) is 2.409 per clip, against 3.182 for direct audio-to-image generation or audio captioning and 2.955 for showing nothing ($p < 0.001$ and $p = 0.044$; 0.087 for the second after a Holm correction over five tests).

Most remaining errors are sounds the detectors barely hear under speech or music, and a vision model that confuses “the object is in the frame” with “the object is making this sound now”. Every decision was made on the development set and the test set was only scored; no study with deaf viewers was run.

Contents

1	Introduction	3
1.1	Motivation and the gap	3
1.2	Problem	3
1.3	Why ready-made models are not enough	3
1.4	Research questions	4
1.5	Contributions	4
2	Related work	5
3	Task and metric	6
3.1	Terminology	6
3.2	Needed sounds	6
3.3	Scoring a picture	7
3.4	Viewer cost	7
3.5	Statistical tests and decision rules	7
4	Data	8
5	The system	9
5.1	Overview	9
5.2	Models at a glance	9
5.3	Detection	10
5.4	The on-screen check	11
5.5	Pictures and display	11
6	Results	12
6.1	Baselines	12
6.2	Main result	12
6.3	What the statistical tests support	14
6.4	Sensitivity to the cost of a wrong picture	15
6.5	Where the errors come from	16
7	Why the problem is hard for today’s models	17
7.1	The scale of the search	17
7.2	Hearing quiet sounds	17
7.3	Confirming weak detections	19
7.4	Presence versus production	19
7.5	Additional vision tools trade one error for another	20
7.6	Answer-form biases	21
7.7	Drawing one unseen source	21
7.8	Automatic evaluation	21
7.9	Scarcity of suitable data	21
7.10	What today’s models are missing	21
8	Error analysis	22
8.1	Ceiling analysis	22
8.2	The on-screen check in detail	23
8.3	Label strictness	23
8.4	Examples	23
9	Limits, threats to validity and future work	24
9.1	Limits of the approach	24
9.2	Limits of the evaluation	25
9.3	Threats to validity	25
9.4	Future work	25
10	Conclusion	26
	Code and data availability	27
	References	27
A	Prompts used in the system	31
B	System details and configuration	32
C	Reproducibility and sources of the numbers	37
D	Glossary of internal names	39
E	Development history	41

1 Introduction

1.1 Motivation and the gap

Deaf and hard-of-hearing (DHH) viewers receive speech through subtitles. The rest of the soundtrack is mostly lost. Captions for the deaf may name a sound in brackets, but the DCMP Captioning Key [15] tells captioners that a sound’s source “may be omitted if it can be clearly seen onscreen”, and a study of more than 715,000 YouTube videos found hand-written non-speech captions in only 5–7% of them [42]. DHH users want to know about the sounds that carry information – knocks, alarms, sirens, a baby crying [31, 33] – and want their type and timing [43]. Among the needed sounds of the benchmark of this report, 30 of 123 (about a quarter) are danger sounds or key moments (importance 3, Section 3.2).

What a viewer loses therefore depends on whether the source is visible. A siren over a visible ambulance tells the viewer nothing new. A siren over a crowd looking up *is* the scene. This project targets the second case, and no existing channel covers it. Subtitles carry speech only. Hand-written sound captions are rare, and the captioning rules say which sounds to describe but give no tool to find them. Sound-awareness apps for DHH users [33, 32] alert to sounds in the room, not in a recorded video, and do not ask whether the viewer can already see the source. Automatic audio captioning [17, 37, 18] describes the whole soundtrack, visible and invisible alike. A system that shows exactly the sounds the viewer cannot see, at the moment they happen, does not exist (Section 2).

1.2 Problem

Given a video, produce the same video with a side panel that shows a picture of a sound *while* the sound is heard and its source is *not* on screen, and shows nothing otherwise. Speech and music are never drawn (speech has subtitles, and music is marked in ordinary captions); human sounds such as laughter, coughing or a scream are drawn. The output is a picture rather than a word in brackets because a picture can be taken in at a glance, without reading, while the viewer is already reading subtitles; this is the design hypothesis behind RQ3 (Section 1.4), and it is not tested with viewers here. No model may be trained: the 159 labelled clips are far too few to train on, and all of them are needed for evaluation. The system must work on one video at a time, as a captioning tool does when a video is uploaded. For every sound this means four questions:

1. **Is there a sound, and when does it start?**
2. **What is it?** At a level a picture can show: *dog bark*, not *animal*.
3. **Can the viewer already see it?** If the thing making the sound is visibly making it, stay silent.
4. **What should the picture show?**

1.3 Why ready-made models are not enough

At first sight the task looks like a short chain of ready-made models: a sound tagger names the sound, a vision model says whether its source is on screen, an image model draws it. In practice every link of that chain fails in its own way, and the failures add up:

- **Hearing.** Taggers rank *Speech* or *Music* above a quiet door or hammer, so the sound never becomes a candidate; most raw detections are not real sounds.
- **Confirming.** An audio-language model asked “is there a siren?” is right only about a third of the time when it alone says yes.
- **Seeing.** A vision-language model answers “is there a bell?” when asked “is the bell ringing?”: it takes presence for production. Every extra vision tool tried (object boxes, crops, segmentation, synchrony, sound localisation) traded one error for another.

- **Drawing.** Audio-to-image models draw the whole soundtrack as one scene and cannot leave out the dog that is already on screen.
- **Scoring.** Model judges and model-written references either reward the system that wrote them or decide visibility at chance, so even measuring progress needs human per-sound labels.

The end-to-end effect is measurable. Even with the full detection stack of the final system, drawing every detected sound without reading the video (the proposal’s direct audio-to-image baseline, Section 6.1) finds 26 needed sounds but shows 62 wrong pictures on the test set, and is no better than showing nothing. Turning the chain into a system that costs less than every baseline took the rules of Section 5, each one answering a failure described in Section 7: two detectors, one of three independent witnesses for every weak sound, a three-question on-screen vote, guarded picture prompts with a check-and-redraw loop, and a per-sound metric to decide each step, found after more than 150 scored variants.

1.4 Research questions

From the project proposal (6 August 2026); Section 10 answers each.

- RQ1. Which types of audio information contribute most to scene understanding when presented visually?
- RQ2. What level of semantic granularity should be used to represent ambient sounds?
- RQ3. Can automatically generated visual augmentations improve accessibility beyond subtitles?
- RQ4. How should semantic audio augmentation systems be evaluated?
- RQ5. How does the proposed pipeline compare with representative audio-to-visual approaches?

1.5 Contributions

1. **The task, a metric and a benchmark.** A precise rule for when a picture is needed, a per-sound score that prices a missed sound against a wrong picture, and 159 video clips with every sound timed, marked visible or not, and rated for importance (Sections 3 and 4).
2. **A system from open models, without training, that costs less than every baseline of the proposal.** On the test set it costs less than both direct audio-to-image generation ($p < 0.001$) and showing nothing ($p = 0.044$, 0.087 after Holm): clearly against direct audio-to-image generation, at the margin against showing nothing. Audio captioning scores as direct audio-to-image generation under the per-sound metric. The system keeps every danger sound that drawing every sound finds (10 of 13) while showing 38 fewer wrong pictures (Sections 5 and 6).
3. **Two findings on why ready-made models are not enough.** Vision-language models take presence for production, and every extra vision tool trades one error for another; and no audio stage tried here separates a right off-screen name from a wrong one well enough to add hits without two to five wrong pictures each (Section 7, with the capabilities today’s models lack in Section 7.10).
4. **A pre-registered method at scale.** 66 pre-registered rounds, each accepting a change only by a rule written down before its run, over more than 150 full-pipeline variants and more than 30 open models (Section 7.1, Appendix E), with a record of every rejected idea and why it failed (Table 14), so the next attempt need not repeat them. No single model swap produced the gain; it is the sum of small rules, each closing one failure.
5. **An evaluation protocol** (the answer to RQ4): per-sound human labels, errors split by cause, and a cost reported over a range of weights (Sections 3, 6.4 and 8); model-written references and model judges were tried and fail at the visibility question (Section 7.8).

How the report is organised. Section 2 reviews related work. Sections 3–5 define the task, the data and the system; Section 6 gives the results, Section 7 explains why the problem is hard for today’s models and Section 8 gives the evidence on the remaining errors; Section 9 lists the limits and threats to validity. The appendices hold the prompts, the detection details and configuration, reproduction steps with the source of every number, a glossary of internal names, and the full development history. The code, labels and report sources are published as release v1.3.0 of the project repository (see Code and data availability).

2 Related work

Non-speech sound for DHH viewers. Captioning practice [15] need not describe a sound whose source is clearly seen on screen; this is the visibility rule used here. Viewers want important sounds, their type and timing, and a choice [43, 2]; captions lose emotional information [57]; visual styling is welcome when it is readable and calm [13, 14]. The closest prior work, Alonzo et al. [2], studied with DHH viewers how non-speech sounds in user-generated videos could be captioned and visualised; it asks how to present the sounds, while this report asks which sounds to show, and when, without a human in the loop. Sound-awareness tools [30, 31, 32, 33, 34] show that users want urgent sounds fast and are bothered by false alarms, which is why a wrong picture has a cost in the metric used here, but none of it selects the sounds of a recorded video by what is already visible.

Sound event detection and audio captioning. Almost all sound taggers use the AudioSet label set [23] (527 classes in a tree, e.g. *Bark* under *Dog* under *Animal*). Clip-level taggers such as PANNs [39], AST [25] and BEATs [10] say *what* is heard. CLAP [20] scores a sound name against a whole clip; frame-level text-queried detectors such as FLAM [68], FlexSED [27] and DASM [8] score it frame by frame. Detection is usually scored with event-based F1 and a time tolerance (“collar”) [44, 5]; the metric used here keeps the onset tolerance but reports a cost instead of F1, in the spirit of cost curves [19]. A known weakness matters here: a quiet sound under speech or music is ranked below *Speech* or *Music*. Automated audio captioning [17, 37, 18] describes a whole clip in a sentence, without asking whether a source is visible. Dense audio-visual event benchmarks such as UnAV-100 [24] label events that are both heard and seen, the opposite of the selection needed here. Audio-visual video parsing [64] (the LLP dataset) comes closest: it labels each segment of a video as audible, visible or both. But “visible” there means that the object appears in the frame, not that it is seen producing the sound, which is the distinction this task turns on.

Audio-language and vision-language models. Audio-language models such as Qwen2-Audio [12], Qwen3-Omni [69] and Audio Flamingo Next [48] answer questions about audio; they name sounds in free text but give no calibrated confidence (Section 7.10). Vision-language models such as LLaVA [40] (not used here), Qwen2.5-VL [4] (used in early versions) and Qwen3.8 [53] (used in the final system) answer questions about frames. Among the tools tried here, vision-language models were the only general means of deciding, for any sound, whether its source is visible. Their known failures matter: a preference for some answer letters [72], weak handling of negation [1], and treating a visible object as the one that is sounding (Section 8), a failure no benchmark measures although it is the central question of this task.

On-screen sound, localisation and audio-to-image generation. Self-supervised audio-visual models [50, 65] learn to separate on-screen from off-screen sound; they separate the audio signal and do not name or depict the off-screen source. Localisation models [59, 66] find *where* in the frame a sound comes from, but localisation maps change little when the audio is silent or off-screen, so these models still point at a visible object [36]; they do not answer *whether* the source is visible. Synchronisation models such as Synchformer [28] estimate whether the soundtrack and the frames are in step; they assume the source is in view and do not say whether it is. Audio-to-image models [62, 6, 22] draw a picture from a whole soundtrack, so they draw the scene rather than the

one unseen source and cannot be told to leave a visible object out. For the same reason they cannot serve as a per-sound baseline: their output has no start time and no single sound that could be matched to a labelled one (Section 6.1). A text-to-image model (Qwen-Image [67, 54]) with a short, guarded prompt is therefore used, and the proposal’s direct audio-to-image baseline is represented by the final system drawing every detected sound, without the on-screen check (Section 6.1).

The gap. The nearest neighbours each cover one part of the task: captioning guidelines and DHH studies say which sounds viewers need [15, 2, 43]; sound event detectors and audio captioning name what is heard; audio-visual parsing marks a segment visible when the object appears; on-screen separation and localisation models relate a sound to the frame; audio-to-image models draw what is heard. What is new in kind is the needed-sound rule as a per-sound, timed, scorable decision. The principle is the captioning guideline’s, but no work reviewed here defines “seen making it now” as the test, adds the obvious and importance tests, or scores a system sound by sound against it, with a viewer cost that prices a miss against a wrong picture; the benchmark of this report does all three. What is new in combination is the system: existing open detectors, listeners, a vision-language model and an image model, chained so that each one checks the failure of the one before, with every weak sound needing an independent witness and every stretch a vote on whether the source is seen making the sound.

3 Task and metric

3.1 Terminology

Final system the system described in this report (release v1.3.0 of the code).

Direct audio-to-image the baseline that draws every detected sound without checking whether its source is on screen (Section 6.1).

Show nothing a system that never shows a picture.

Gate (on-screen check) the step that decides whether a sound’s source is visible.

Listener an audio-language model asked about a sound (“is a siren present?”, “list what you hear”).

Family two labels are in the same family if they are equal or one lies under the other in the AudioSet tree (*Bark* and *Dog*), and neither is a vague label near the top of the tree such as *Animal*.

Span the time interval of one detected sound.

Stretch a piece of a span, about 5 s long; the on-screen check judges each stretch separately.

Bootstrap resampling the clips many times to see how much a difference could change by chance (defined in Section 3.5).

Holm correction a stricter p -value bar used when several comparisons are made at once (Section 6.3).

3.2 Needed sounds

The unit of evaluation is one sound in one clip. For every sound the annotator records its label, its start and end (about 0.5-s precision) and three judgements:

visible the thing making the sound is *seen making it now*. A barking dog on screen is visible; a baby held quietly is not visibly crying; an alarm box on a wall is not visibly ringing.

obvious with the sound off, a viewer would still know it is happening (waves when the sea fills the frame).

importance rated as if the screen were black: 1 = steady background noise; 2 = an event one can say in a sentence; 3 = danger or a key moment (siren, alarm, gunshot, glass, baby crying, scream).

A sound is **needed** if it is neither visible nor obvious and has importance 2 or 3. *Example:* a police siren is heard while the camera shows a crowd looking up – needed. When the police car drives into view with its lights on, the siren becomes visible – not needed. Every heard sound is labelled, because a picture over an unlabelled moment counts as wrong.

3.3 Scoring a picture

Pictures are scored as they are shown on screen. A picture **hits** a needed sound if it is of the same family and *starts* between 0.5s before and 1s after the sound starts; each sound takes at most one picture. Every other picture is **wrong**, of one of three kinds:

on screen it shows a sound whose source is visible or obvious (the viewer could see it or already knows it);

other sound it shows the wrong sound – something else is heard there (a gunshot picture over footsteps), including a steady background sound of importance 1, or it is a late second picture of a long sound already shown;

nothing there is no labelled sound there at all.

“Wrong” is therefore strict: it includes redundant pictures, not only false ones. The hit window is asymmetric on purpose: a picture that appears slightly before its sound is still read as that sound, while a picture that appears more than a second late has lost its point, because the moment has passed. The window was fixed in advance and is the same for every system. Three edge rules complete the scoring. A picture of an off-screen sound of importance 1, starting in its window, is neither a hit nor wrong. A picture that matches both a visible and a needed sound of its family is a hit. One picture may hit several overlapping sounds of the same family. Each of these cases occurs once in the final system.

3.4 Viewer cost

The main score is the cost per clip,

$$C = \frac{4 \cdot \# \text{missed needed sounds} + 2 \cdot \# \text{wrong pictures}}{\# \text{clips}}, \quad (1)$$

lower is better. A miss costs twice a wrong picture, so one right picture pays for two wrong ones, and a picture is worth showing when it is right at least one time in three. These weights are an assumption, not a measurement, and were fixed early in the project, so Section 6.4 also reports other weights, including a version in which an on-screen picture costs only 1 or 0 (it is less harmful than a picture of nothing):

$$C_w = \frac{4 \cdot \# \text{missed} + w \cdot \# \text{on screen} + 2 \cdot \# \text{other sound} + 2 \cdot \# \text{nothing}}{\# \text{clips}}, \quad w \in \{0, 1, 2\}. \quad (2)$$

Why not F1? Event-based F1 is the usual detection score [44, 5], but it counts a miss and a wrong picture the same, while the gate’s value lies almost entirely in the wrong pictures it removes. A cost with explicit error prices, as in cost curves [19], makes that trade-off visible. F1 is reported alongside the cost (Table 3); the cost is primary.

3.5 Statistical tests and decision rules

Paired bootstrap. Comparisons are paired by clip and tested with a paired clip bootstrap. For each clip the difference in cost between the two systems is computed; the clips are then drawn

with replacement 100,000 times (random seed 0), and the mean difference is recomputed for each draw. The 95 % interval runs from the 2.5th to the 97.5th percentile of these means. The two-sided p -value is twice the share of draws in which the difference is zero or has the opposite sign; $p < 0.001$ is printed when fewer than 1 draw in 2,000 did so; in both cases reported here, no draw did. The bootstrap is used for the comparisons reported in Section 6. Differences are always final minus baseline, so a negative cost difference favours the final system.

Pre-registration. With sets this small, a rule chosen after seeing its result could fit noise. A change to the system was therefore accepted only if it passed a rule written down *before* its run, on the development set (the rule and its pass bar were committed to the repository first; this is called pre-registration). It passed if either

- (a) it kept or raised the hits, added at most two wrong pictures per extra hit, and lowered the cost; or
- (b) it lost at most three hits and removed at least three wrong pictures per hit lost.

These pass rules are count rules on the development set, not significance tests. Rules about the audio alone could first be screened on the screening set (Section 4). The test set was scored only after a change was accepted, and only reported; no decision in the final system used it. The 95 % intervals of the main comparisons are 0.5 to 1.2 per clip wide on both sets, so small single changes could only be judged on the development rules.

4 Data

All results use two sets of short video clips (10–20s), labelled per sound as in Section 3.2:

Development set 71 clips, 58 needed sounds. Every design decision was made on this set.

Test set 88 clips, 65 needed sounds. Scored only after a change was accepted, and only reported. The clips come from Wikimedia Commons, the Internet Archive, UnAV-100 [24] (its videos are used as scene material; its labels are not used), VGGSound [9], YouTube (city walks, markets, home, transport, incidents, film scenes) and the AudioSet-Strong evaluation split (AudioSet clips that come with timed, per-sound labels). New clips were assigned to development or test by a hash of the clip name or in pairs matched by scene, one to each set, never by looking at results; Appendix E.1 describes how the sets grew. Most clips are copyrighted and used for research only, so they are not redistributed (see Code and data availability).

Table 1: The two evaluation sets.

Set	Clips	Labelled sounds	Needed sounds	Clips with a needed sound
Development	71	154	58	38
Test	88	154	65	43

About half of the clips have no needed sound; on those, the best possible behaviour is to show nothing. Three needed development sounds (a clang and two golf-club strikes) belong to labels the system never draws (the label filter removes generic impact sounds, Appendix B), so they are misses for every system.

Label quality. All labels are the author’s; a second annotator was planned but did not return labels, so inter-annotator agreement is unknown. On the earlier clip-level labels, a blind relabelling by the author agreed 78 % of the time (Cohen’s $\kappa = 0.60$; κ discounts chance agreement, 1 is perfect, 0 is chance). That is the author’s agreement with their own earlier clip-level labels, not per-sound agreement with a second annotator. Two blind checks of pictures counted as wrong found that

about 1 in 6 is a real off-screen sound the labels miss (Section 8.3). These and other caveats are listed in Section 9.

Screening set. 415 clips of the AudioSet-Strong evaluation split, which already have timed labels, were used to screen rules about audio alone (for example: how often is a detection that no listener names a real sound?). AudioSet labels only a few sounds per clip, so this set was never used to accept a change.

5 The system

This section describes the final system as the released code runs it. Appendix B gives every step and threshold.

5.1 Overview

Figure 1 shows the data flow and Figure 2 an output. For one video (stage numbers as in Figure 1):

- **Hear candidates (4a).** Two sound detectors listen to the whole clip: BEATs (a tagger for the 527 AudioSet classes) and FlexSED (a detector that is asked one sound name at a time). Anything either hears strongly enough for at least 0.3 s becomes a candidate with a start and an end.
- **Check candidates (4b).** A candidate heard by only one detector, or heard weakly, is doubtful. Two audio-language models (the “listeners”) and a third detector give second opinions. Two kinds of rule use them: a *veto* drops a candidate that another audio model does not hear; a *rescue* keeps a weak candidate that a listener names. The models are listed in Table 2.
- **Witness rule (4b).** A remaining weak candidate survives if one of three witnesses backs it: DASM [8], a third detector, clearly hears it at that time; or both listeners list it; or one listener lists it and the vision model says the sound fits the scene.
- **Refine (4c).** The start of each BEATs sound is moved to where its evidence begins (never earlier than detected), and rescued sounds pass three last filters: FineLAP and DASM must hear them, and only one rescue per family is kept.
- **On-screen check, the gate (5).** A vision-language model (Qwen3.8-27B) looks at frames around the sound. If the thing making the sound is visibly making it, no picture is shown.
- **Choose the subject (5b).** A sound is also silenced when a more specific sound of its family is visible (kinship). For each remaining sound the vision model chooses the subject phrase, 2–6 words saying what to draw.
- **Draw (6).** A short, guarded prompt is written for each remaining sound; Qwen-Image draws it; the vision model checks that the picture shows the intended thing and it is redrawn up to five times.
- **Clean the display (6b).** Repeats less than 2.5 s apart become one picture. A listener decides whether two same-family pictures up to 8 s apart are one continuing sound. A picture is also dropped when the vision model says the drawn event itself (“a door slamming”) is visibly happening; this second check asks about the event, not its source.
- **Compose (7).** Pictures appear in up to three fixed slots beside the video while their sound plays (at least 1.5 s).

5.2 Models at a glance

Table 2 lists the model and the main settings of every stage.

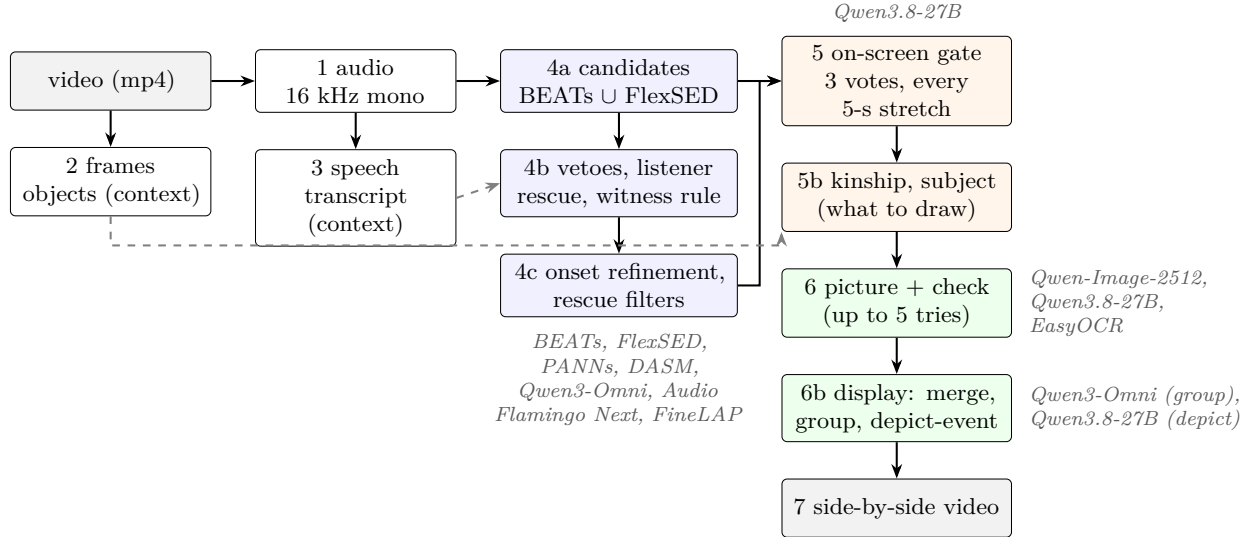


Figure 1: The final pipeline. Blue: detection (stage 4); orange: the on-screen decision (stage 5); green: pictures and display. Dashed: context only (objects and transcript never decide on their own). The scene check of the witness rule in stage 4b also uses Qwen3.8-27B. Vetoes: rules that drop a candidate when another audio model does not hear it. Rescue: a listener adding a sound the detectors almost missed. Kinship: a sound is also silenced when a more specific sound of the same family is visible. Depict-event: dropping a picture whose event is visibly happening.

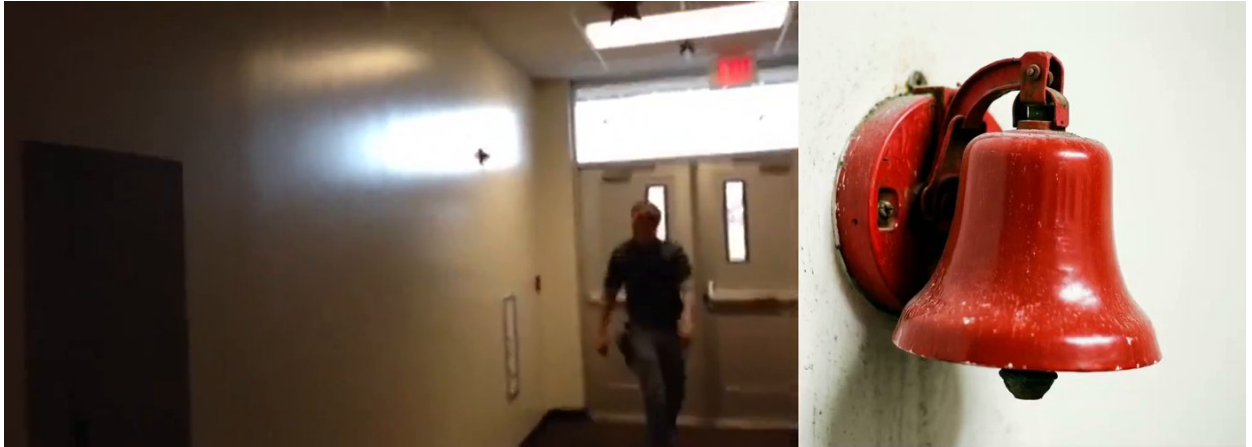


Figure 2: Output of the final system on a development clip (AudioSet-Strong, a corridor). A fire alarm rings from 9.0s; the alarm bell itself is not in the frame. The system detects *Alarm*, the on-screen check finds no visible source, and the panel on the right shows the generated picture from 8.9s while the alarm sounds. In the scoring this is a hit.

5.3 Detection

The two detectors complement each other: BEATs is strong on clear sounds, FlexSED hears quiet sounds under speech and music far better. Most of the development effort went into deciding which weak candidates to keep. The listeners are asked about each weak candidate in two ways – a yes/no question (“Is the sound of a siren present in this recording?”) and an open list (“List every distinct non-speech sound you hear”). The witness rule rests on how often the listeners agree (Section 7). Speech and music are never drawn; human sounds and all other sound families with a clear source are (Appendix B).

Table 2: Stages, models and key parameters of the final system.

#	Stage	Model / tool	Key parameters
1	audio	ffmpeg	mono WAV, 16 kHz (<code>SAMPLE_RATE</code>)
2	frames, objects	OWLv2 object detector [47] (<code>owlv2-base-patch16-ensemble</code>)	6 frames, threshold 0.20; context only, never silences a sound
3	speech	faster-whisper [63] base (Whisper [56]), int8	transcript used only to ask whether people react to a sound; never drawn
4	sound events	BEATs iter3+ AS2M [10]; FlexSED [27]; PANNs CNN14 [39]; DASM [8]; Qwen3-Omni-30B-A3B [69]; Audio Flamingo Next [48]; FineLAP [3]	16 ordered steps, Table 7
5	on-screen gate	Qwen3.8-27B [53], greedy, thinking off	6 frames per ≤ 5 -s stretch, 3 votes, majority; silenced only if every stretch is visible
5b	subject	Qwen3.8-27B + word lists	2–6 word phrase; speech text never used
6	picture	Qwen-Image-2512 [67, 54]	1024×1024 , 50 steps, true CFG 4.0; check-and-redraw ≤ 5 tries; EasyOCR [29] finds text, and a picture with text is redrawn
6b	display	Qwen3-Omni (group), Qwen3.8-27B (depict)	merge gap 2.5 s, min. 1.5 s on screen, ≤ 1 s past the end, group ≤ 8 s, 3 slots
7	composite	ffmpeg libx264	720 px panel, 25 fps

Greedy: always the most likely next word. Thinking off: no hidden reasoning text. int8: reduced numeric precision for speed. CFG: how strongly the picture follows the prompt.

5.4 The on-screen check

For every detected sound, the gate cuts its time span into stretches of about 5 s, shows the vision model six frames from each stretch and asks three questions (exact prompts in Appendix A):

1. **Name it:** “Name the thing in these frames that is making that sound, or answer: nothing.” If the answer shares a word with the label (“church bell” for *Bell*) this vote says visible; otherwise a text-only follow-up asks whether that thing could make the sound.
2. **Choose:** “(a) you can see it happening on screen, or (b) it is not visibly happening”, asked in both orders; if the two answers disagree the vote abstains.
3. **Describe:** a one-sentence description of the frames, checked against the sound in the same way.

A stretch counts as visible if most votes say so, and the sound is silenced only if *every* stretch is visible. This leans toward showing: a source that leaves the frame for one stretch keeps its picture.

5.5 Pictures and display

The picture prompt is built from the detected label, not from free text: the vision model may add at most two qualifier words to the source’s name, word lists remove people and places, and an action is drawn with its maker (a *bark* becomes a barking dog). A fixed tail asks for one large subject on a plain white background. Qwen-Image-2512 draws it at 1024×1024 ; the vision model then gets a multiple-choice question about what the picture shows, and a picture that fails (or contains text) is redrawn, up to five times. In a blind test, the author saw each picture for a moment and named the sound without being shown the label (one test, by the author only); Qwen-Image pictures were named right 26 of 54 times, against 14 of 54 for the earlier FLUX.1 [7] pictures (difference +0.22,

95 % interval $[+0.07, +0.37]$). On screen, repeats of one sound less than 2.5 s apart become one appearance, every picture stays at least 1.5 s and at most 1 s after its sound ends, and at most three pictures are shown at once.

The scored numbers come from a scoring program (the “harness”) that re-runs the pipeline’s own decision functions on saved model outputs; an independent re-export of every decision reproduces them exactly. The live pipeline differs in four small ways that do not change when a picture starts (Appendix B).

6 Results

6.1 Baselines

The proposal names three baselines. All are run on exactly the same clips and scored with exactly the same rules:

Direct audio-to-image generation is the final system with the on-screen check switched off: the same detection stack, listeners, display rules and picture model, with a picture for every surviving sound. The other steps that read the video to decide visibility, chiefly the depict-event check, are switched off with it (Appendix B.4). Because everything else is held fixed, the comparison isolates the question the proposal asks: what reasoning about what the video already shows adds to the same detections. (Published audio-to-image models draw a whole scene from the soundtrack and cannot be scored per sound; Section 2.)

Audio captioning describes the same sounds in text instead of pictures. Under the per-sound score a caption is right or wrong at exactly the same moments as the audio-to-image picture, so it has the same score; the difference between text and pictures can only be measured with viewers (Section 9).

Show nothing no augmentation at all: the situation of a viewer with subtitles only.

A generator conditioned on the video and audio that would “generate what is not seen” (for example MiniMax-H3) was also raised during supervision. It produces whole new video, which the per-sound score cannot evaluate, so it was not run as a baseline (Section 9).

6.2 Main result

Figure 3 and Table 3 give the main result.

Table 3: Main result on the development set (71 clips, 58 needed sounds) and the test set (88 clips, 65 needed sounds). Hits: needed sounds with a right picture. Wrong: all other pictures, split into on screen / other sound / nothing (Section 3.3). Cost: viewer cost per clip, $(4 \times \text{misses} + 2 \times \text{wrong}) / \text{clips}$, lower is better. Difference: the final system’s cost minus this system’s, per clip, with a 95 % interval from a paired clip bootstrap (100,000 draws, Section 3.5) and a two-sided p -value, uncorrected.

Set	System	Hits	Wrong	F1	Cost	Difference (final – this) [95 % CI]	p
Development (71 clips)	Show nothing	0 / 58	0	0.00	3.268	−1.211 [−1.831, −0.620]	< 0.001
	Direct audio-to-image ^a	34 / 58	39 (20/14/5)	0.52	2.451	−0.394 [−0.817, +0.056]	0.085
	Final system	29 / 58	15 (6/7/2)	0.57	2.056		
Test (88 clips)	Show nothing	0 / 65	0	0.00	2.955	−0.545 [−1.091, −0.023]	0.044
	Direct audio-to-image ^a	26 / 65	62 (16/32/14)	0.34	3.182	−0.773 [−1.045, −0.500]	< 0.001
	Final system	24 / 65	24 (4/15/5)	0.42	2.409		

^a The final system without the on-screen check; audio captioning has the same score (same decisions, shown as text). With the Holm correction over five tests (Section 6.3) the four p -values of the Difference column become, top to bottom, < 0.001, 0.087, 0.087 and < 0.001.

On the test set the final system finds 24 of 65 needed sounds, and half of its pictures are right. Drawing every detected sound finds 26, but only 30 % of its pictures are right, and on the test

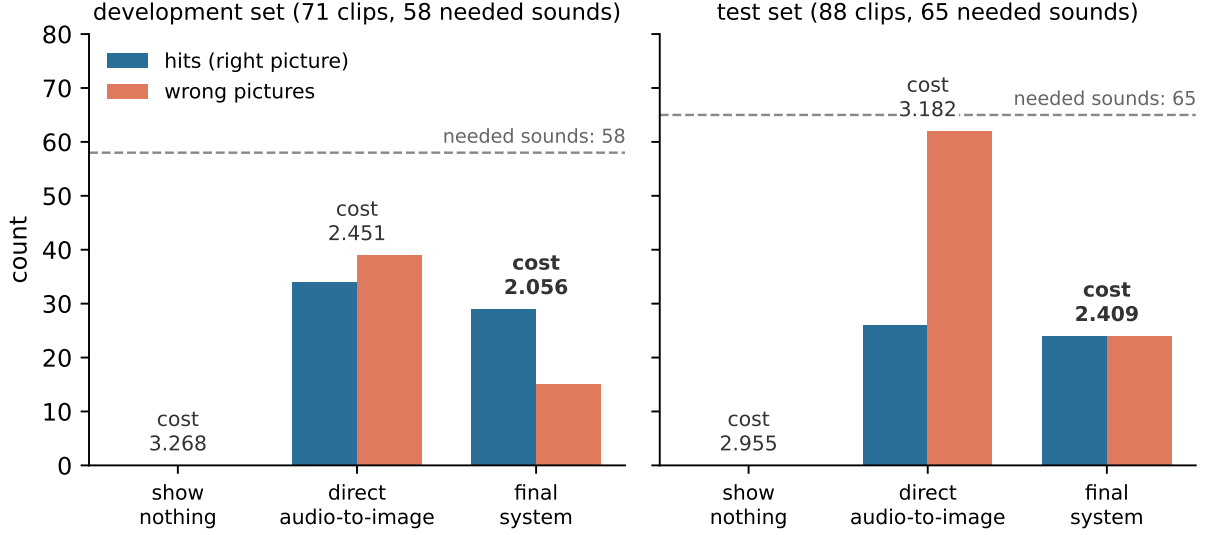


Figure 3: Hits (right pictures of needed sounds) and wrong pictures for each system, with the cost per clip above each group (lower is better). Drawing every detected sound finds a few more needed sounds but shows two to three times as many wrong pictures; the final system keeps most of the hits and removes most of the wrong pictures.

set it is no better than showing nothing: it costs 0.227 more per clip ($[-0.409, +0.841]$, $p = 0.49$, uncorrected). On the development set it costs less than showing nothing ($[-0.817, [-1.577, -0.085]]$, $p = 0.032$, uncorrected). The final system is cheaper than both: by 0.773 per clip than direct audio-to-image generation ($p < 0.001$), which is 24 % of that baseline’s cost, and by 0.545 than showing nothing ($p = 0.044$), 18 % of its cost.

On the development set the direction is the same, but the difference from direct audio-to-image is smaller (-0.394 , $p = 0.085$; 0.087 after Holm). Two things differ between the sets. Direct audio-to-image shows fewer wrong pictures per clip on the development set (39 over 71 clips, 0.55 per clip, against 62 over 88, 0.70), mostly because it shows fewer other-sound pictures there (14 against 32). And the check costs more hits there: five fewer than direct audio-to-image on the development set, but only two fewer on the test set.

The win over direct audio-to-image generation comes from giving up a few hits to remove many wrong pictures. On the test set the final system has 2 fewer hits and 38 fewer wrong pictures: $(2 \cdot 38 - 4 \cdot 2)/88 = 0.773$ per clip, or 19 wrong pictures removed per hit lost. On the development set it has 5 fewer hits and 24 fewer wrong pictures: $(2 \cdot 24 - 4 \cdot 5)/71 = 0.394$, or 4.8 per hit lost. Of the 38 wrong pictures removed on the test set, 12 are on-screen pictures, 17 show another sound and 9 show nothing; on the development set the 24 removed are 14, 7 and 3 (Figure 4). Silencing a sound also removes its late repeat pictures, and the depict-event check, which is off in the baseline, removes some more. Figure 5 shows the two effects on the cost.

F1, which weighs a miss and a wrong picture equally, is also higher for the final system on both sets: 0.57 against 0.52 on the development set ($+0.050$, $[-0.039, +0.128]$) and 0.42 against 0.34 on the test set ($+0.085$, $[+0.032, +0.142]$, $p = 0.002$). The intervals use the same clip resampling, with F1 recomputed from the resampled counts on each draw. F1 favours the final system even though it values a removed wrong picture no more than a lost hit.

Danger sounds. The sounds that matter most to a deaf viewer are the importance-3 sounds: danger or a key moment, such as a siren, an alarm, glass or a baby crying (Section 3.2). On the test set the final system keeps every danger sound that drawing every sound finds (10 of 13) while showing 38 fewer wrong pictures; on the development set it finds 14 of the 17, one fewer (Figure 6). The misses are mostly ordinary events of importance 2.

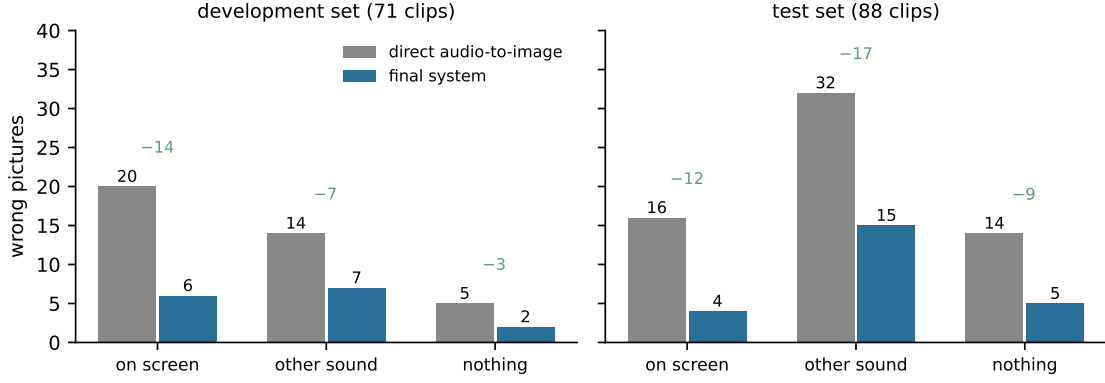


Figure 4: Wrong pictures by kind (Section 3.3), direct audio-to-image generation against the final system. Green: the number removed. Totals: 39 → 15 (development), 62 → 24 (test). The on-screen check removes on-screen pictures directly; silencing a sound also removes its late repeats (other sound), and the depict-event check removes some more. On the test set 17 of the 38 removed pictures are of another sound and 9 of nothing.

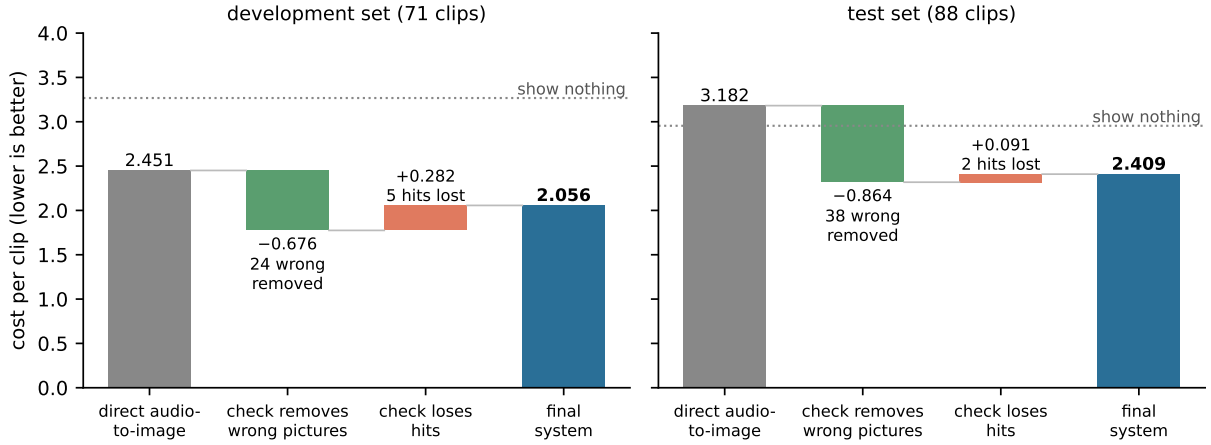


Figure 5: Where the gain over direct audio-to-image generation comes from. Starting from its cost (left), the steps that read the video remove wrong pictures (green, saving 2 per picture) and lose a few hits (red, costing 4 per hit). On the test set they remove 38 wrong pictures and lose 2 hits; on the development set 24 and 5. Dotted line: showing nothing.

6.3 What the statistical tests support

Significant. On the test set the final system costs less than direct audio-to-image generation and audio captioning ($p < 0.001$), has a higher F1 than them ($p = 0.002$), and costs less than showing nothing ($p = 0.044$ uncorrected; 0.087 after the Holm correction below). On the development set it costs less than showing nothing ($p < 0.001$).

Not significant. On the development set, the difference from direct audio-to-image generation ($p = 0.085$).

Multiple comparisons. The family is every comparison of the final system against a baseline for which a p -value is reported: cost against each baseline on each set, and F1 against direct audio-to-image generation on the test set. The two tests of one baseline against the other are descriptive. With a Holm correction over the five, the test-set cost difference from direct audio-to-image generation and the development-set difference from showing nothing stay at $p < 0.001$, the test-set F1 difference becomes $p = 0.007$, the test-set difference from showing nothing becomes $p = 0.087$, and the development-set difference from direct audio-to-image

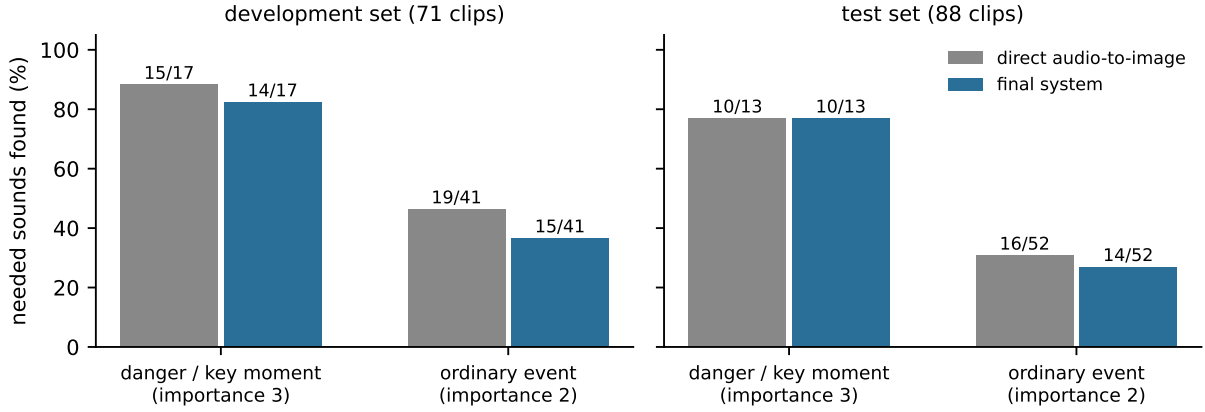


Figure 6: Needed sounds found, by importance level (bars: share found; labels: found / needed). Importance 3: danger or a key moment, such as a siren, an alarm, glass or a baby crying; importance 2: an ordinary event one can say in a sentence. The final system finds 82% (development) and 77% (test) of the danger sounds and 37% and 27% of the ordinary events; direct audio-to-image finds 88% and 77%, and 46% and 31%, with far more wrong pictures (Figure 3).

generation becomes $p = 0.087$.

Reproduced. Every count was reproduced by two independent routes, the scoring harness and the logged decision trail, with no difference on any of the 159 clips (Appendix B.3).

6.4 Sensitivity to the cost of a wrong picture

The cost of one wrong picture (β , with a missed sound fixed at 4) is the one number in the metric that only deaf and hard-of-hearing viewers can set. A viewer study to measure it was designed but not run, so the main analysis assumes $\beta = 2$. Figure 7 shows how the conclusions change if β is different.

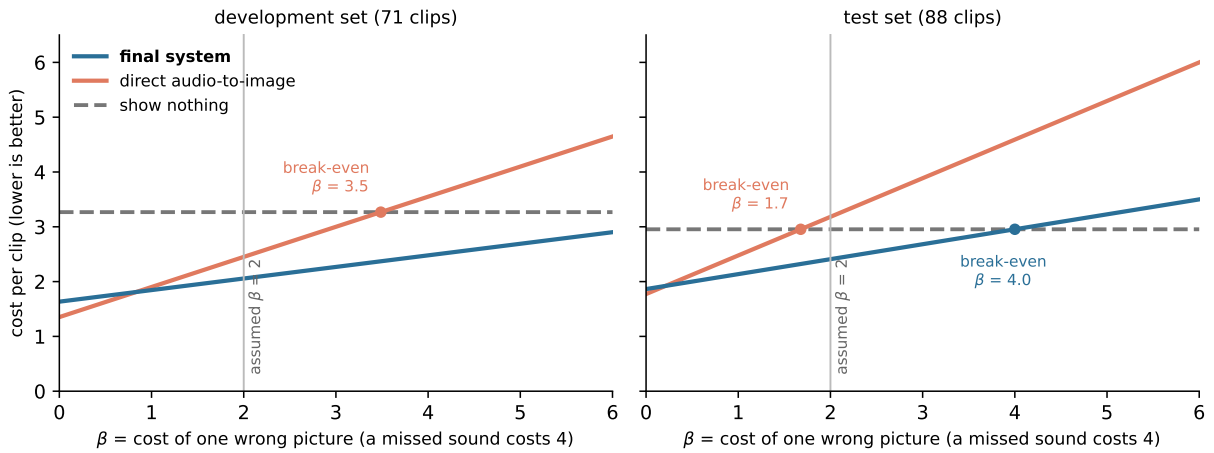


Figure 7: Cost per clip as the cost of a wrong picture (β) varies, with a missed sound fixed at 4 (point estimates). Dots mark where a system breaks even with showing nothing: the final system at $\beta = 4.0$ (test) and 7.7 (development, off the chart), direct audio-to-image generation at $\beta = 1.7$ (test) and 3.5 (development). With 95% bootstrap intervals, the final system is cheaper than showing nothing up to about $\beta = 2.05$ (test) and $\beta = 4.4$ (development).

On point estimates, the final system is cheaper than showing nothing for every $\beta < 4.0$ on the test set, that is, until a wrong picture hurts as much as a missed sound, and cheaper than direct audio-to-image generation for every β above 0.2 (test) or 0.8 (development). With intervals,

recomputed by the same bootstrap at each β , the 95 % interval of the test-set difference from showing nothing excludes zero up to about $\beta = 2.05$; on the development set it excludes zero up to about $\beta = 4.4$. Drawing every sound is therefore only preferable if wrong pictures are almost free. A second check weighs only on-screen pictures less (Equation 2). With $w = 1$ the final system costs 1.972 (development) and 2.364 (test), against 2.169 and 3.000 for direct audio-to-image; with $w = 0$ it costs 1.887 and 2.318, against 1.887 and 2.818. So if an on-screen picture is judged harmless, the two systems tie on the development set, and the final system still wins on the test set.

6.5 Where the errors come from

Figure 8 breaks down the misses of the final system by the step where each was lost, from its logged decision trail. Most misses are sounds that were heard as candidates and then removed by a later rule – on the development set most often because the listeners did not confirm a weak detection; on the test set listeners, vetoes and timing account for about equal shares. Each of these rules was accepted because, overall, it removed more wrong pictures than real ones; the sounds it removes are the price. Six needed sounds in each set were heard by no detector. (These counts are for the final system, with 58 and 65 needed sounds; the ceiling analysis of Section 8.1 used an older run with 55.) Most wrong pictures on the test set show another sound (15 of 24; Figure 4). Figure 9 shows the needed sounds found per gold label, with each miss coloured by the step where it was lost: on the development set none of the four needed bird calls is found, two of them lost at the on-screen check, where perched birds are taken as the source (Section 8.2). The two largest families of the test set, machine gun and vehicle horn, are each found 0 of 5 times, for different reasons. Three of the horns are in one clip with continuous traffic: the continuation veto reads each repeated honk as a later piece of the traffic sound already going on; one more horn is lost to timing and one at the on-screen check. Of the machine guns, two are silenced at the on-screen check, one falls to the mirror veto (a helicopter scored higher at that moment), one is lost to timing, and one is merged into a stronger gunshot of the same family. Section 7 explains why these errors are hard to remove.

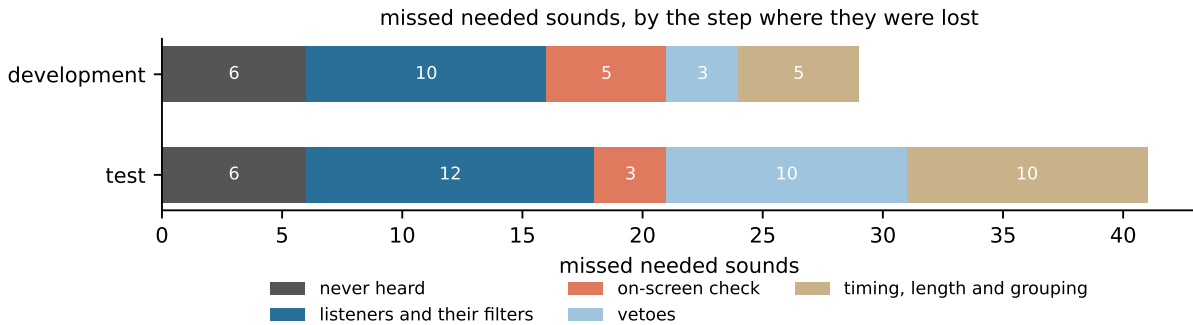


Figure 8: Misses of the final system by the step where the sound was lost (wrong pictures by kind are in Figure 4). Vetoes are rules that drop a candidate because another audio model does not hear it, or because it is a later piece of a sound already going on (the continuation veto); listeners are the audio-language models that confirm or add weak sounds; “timing, length and grouping” covers pictures that start too early or late, sounds too short to show, and sounds merged into a neighbour. The mapping from logged step to bar is given in Appendix C.

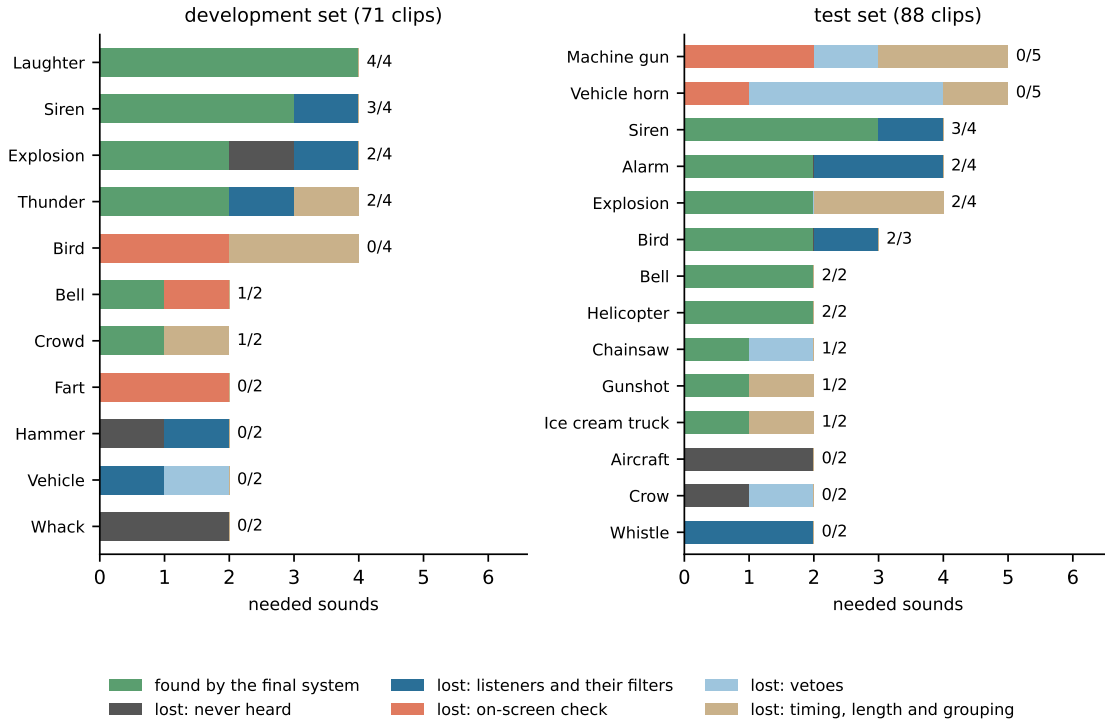


Figure 9: Needed sounds found by the final system, by gold label, for every label with at least two needed sounds in that set. Green: found; the rest of each bar is stacked by the step where the sound was lost, in the colours of Figure 8. Per-sound outcomes from the logged decision trail (`docs/inspector2/data.js`).

7 Why the problem is hard for today’s models

A reader may expect that modern models solve this task directly: a sound tagger hears the siren, a vision model says whether an ambulance is on screen, an image model draws it. This section explains, step by step, why that does not happen. “Hard” here means hard for open models of up to about 70 GB, chained without training; a much larger vision model (Qwen3-VL-235B) was not tested, and fine-tuning was outside the design. Sections 7.2–7.8 each state what one might expect, what actually happens, and the evidence. Table 4 summarises the argument, Section 7.1 gives the size of the search behind it, and Table 14 (Appendix E) lists the main ideas that failed.

7.1 The scale of the search

Table 5 gives the size of the study behind the final system.

Figure 10 shows what the search bought. Each accepted change had to pass its rule on the development set; the cost fell step by step from the first full pipeline, which was worse than showing nothing on the development set and barely better on the test set, to the final system, which is clearly better on both. No single model swap produced this: the gain is the sum of many small rules, each one closing a failure described in this section.

7.2 Hearing quiet sounds

Expected: a modern tagger hears a hammer or a door. *Actually:* under speech or music it often does not. The tagger’s own score for *Speech* or *Music* is higher than for the quiet sound, so the sound never becomes a candidate. A ceiling analysis, in which every later stage is made perfect using the labels, shows that about one needed sound in five stays missed because no model hears it clearly (Section 8.1). Removing the speech first with a source separation model (HT-Demucs [58], keeping its non-vocal “other” stem) made it worse: the share of sounds found under speech fell from 9.5 % to 4.8 %, because the detectors were never trained on separated audio.

Table 4: The obvious ready-made chain, step by step: what goes wrong, and what the final system does instead.

Step	Ready-made choice	What goes wrong (evidence)	What the final system does
Hear	one sound tagger	quiet sounds under speech or music never become candidates; under 40 % of raw detections are real (Section 7.2)	two complementary detectors, with every weak candidate checked
Confirm	ask an audio-language model	a sound one listener names is real 37 % of the time (Figure 11)	a weak sound needs a witness: DASM, both listeners, or one listener plus the scene
See	ask a vision-language model “is it visible?”	presence is taken for production (31 of 36 audited errors); AUROC 0.647; answers follow the question’s form	three questions per 5-s stretch, the a/b question asked in both orders; silenced only if every stretch is visible
Draw	audio-to-image model	draws the whole scene, including what is on screen	label-guarded text prompt; the picture is checked and redrawn up to five times
Score	model judge	circular or at chance on visibility (Section 7.8)	per-sound human labels and a viewer cost
All	full detection stack, every detected sound drawn without reading the video	26 sounds found, 62 wrong pictures on the test set; no better than showing nothing	final system: 24 sounds found, 24 wrong pictures; cheaper than every baseline of the proposal

Table 5: Scale of the study behind the final system (details in Appendix E).

What	How much
Open models evaluated	more than 30: 14 sound detectors and taggers (e.g. BEATs [10], FlexSED [27], PANNs [39], AST [25], EAT [11], Dasheng [16], CLAP [20], FLAM [68], DASM [8], FineLAP [3]); 6 audio-language listeners (Qwen3-Omni [69], Audio Flamingo Next [48], MOSS-Audio [49], SpotSound [41], Kimi-Audio [38], Step-Audio 2 [61]); 2 source separators (HT-Demucs [58], SAM-Audio [46]); 8 vision models and tools (CLIP [55], two Qwen vision-language models [4, 53], OWLv2 [47], SAM 3 [45], SigLIP [71], the sound localiser SSL-SaN [70], Synchronformer [28]); 5 image generators (SDXL [52], SDXL-Turbo [60], FLUX.1-schnell [7], PixArt- Σ [51], Qwen-Image [67]); and 3 model judges [73] (Mistral-7B [35], Qwen3-8B and Gemma-4-31B [26])
Pre-registered rounds	66 rounds, each with its rule and pass bar written before the run
Full-pipeline variants scored	more than 150, plus about 90 component screens
Human labels	308 timed sounds in 159 clips, each marked visible or not and rated for importance
Screening data	415 AudioSet-Strong clips with timed labels for audio-only rules

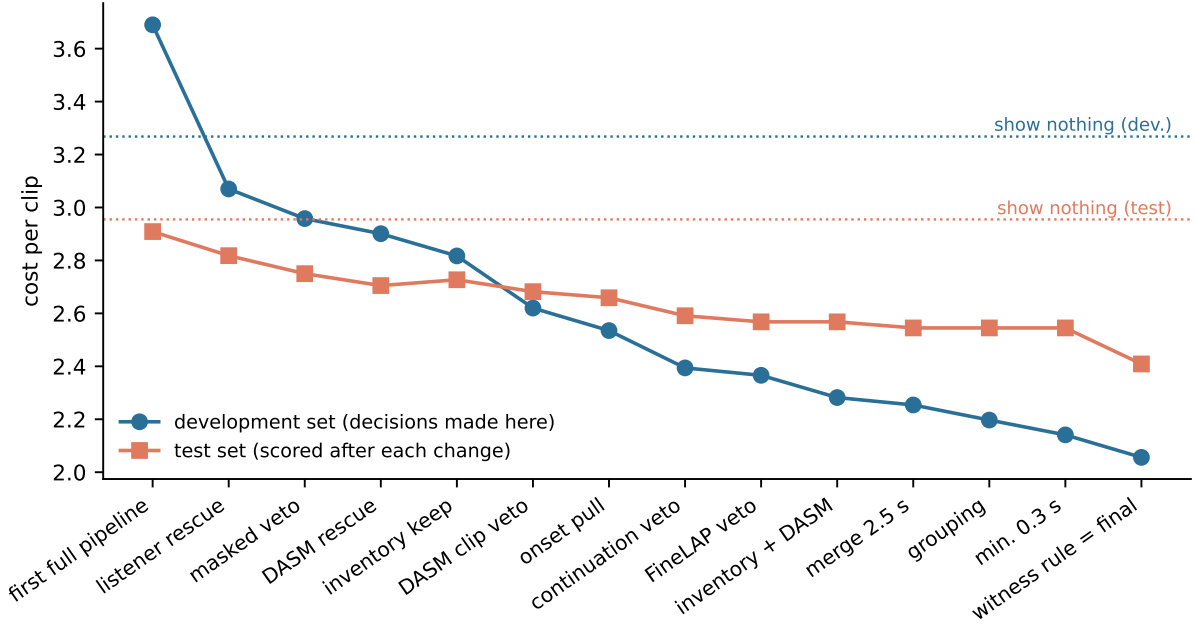


Figure 10: Cost per clip after each accepted change, from the first full pipeline to the final system: development 3.690 $\rightarrow$ 2.056, test 2.909 $\rightarrow$ 2.409. Decisions were made on the development set; the test set was only scored. Dotted lines: showing nothing. Each step is a row of Table 13 (Appendix E), except three early steps (masked-weak veto, DASM rescue, inventory keep), which share one row there; the step names are short forms of the internal names explained in Appendix D.

7.3 Confirming weak detections

Expected: an audio-language model can confirm “is there a siren?”. *Actually:* its answers are only weak evidence. On the screening set, a weak detection that neither listener names is a real sound 22 % of the time, one named by one listener 37 %, and one named by both 62 % (Figure 11; AudioSet labels are incomplete, so these shares are lower bounds, and the ordering, not the level, is the evidence). Raw detections are right under 40 % of the time. A full chain of listener, timing and DASM checks reaches 80 %, but only for the few sounds that survive the chain. Every way to add more sounds found real ones but paid two to five wrong pictures per new hit. The best such rule found two more hits than the final system on the development set, at about five extra wrong pictures each (31 hits and 26 wrong, against 29 and 15), so it failed the development rule; its test result, reported only, was no better (25 hits and 35 wrong, against 24 and 24). No audio stage tried here separates a right off-screen name from a wrong one well enough to add hits without two to five wrong pictures each.

7.4 Presence versus production

Expected: a vision-language model can answer “is this bell ringing?”. *Actually:* it answers “is there a bell?”. A church tower on screen silences a bell heard off screen; perched birds are taken as the source of a call made elsewhere. This is the main cause of the gate’s wrong silences (31 of 36 in a hand audit, Section 8.2). The opposite failure, answering “nothing” when the source is small or hidden (a hand, the footsteps of the person holding the camera), is also large. The failure is not specific to one model: an early image-text matching check (CLIP [55]) was right only about half the time, and moving from a 7-billion- to a 27-billion-parameter vision model raised balanced accuracy (the mean of the success rates on visible and on needed sounds) only from 0.61 to 0.62.

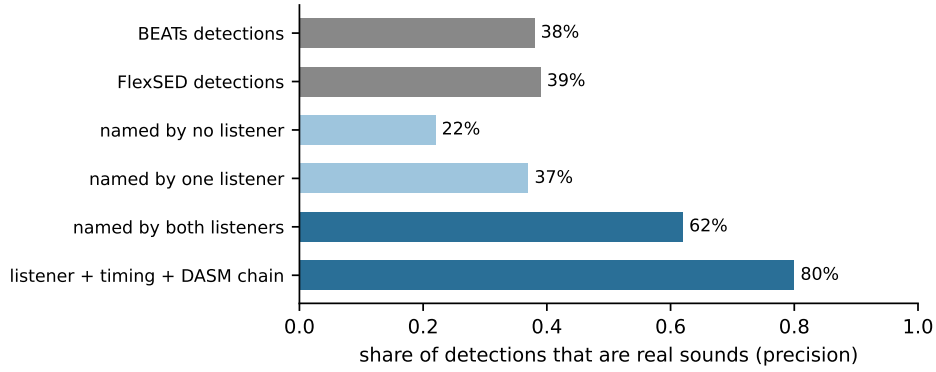


Figure 11: Share of detections that are real sounds, on the screening set (415 AudioSet-Strong clips with timed labels). Raw detections are right less than 40 % of the time; agreement between models helps, but a sound only one listener names is right just over a third of the time. AudioSet labels are incomplete, so all shares are lower bounds. The last bar is the audio chain of a rejected idea (Appendix E), shown for reference.

7.5 Additional vision tools trade one error for another

Expected: adding an object detector, a segmentation model, a crop of the source, a model of audio-visual synchrony or a sound-localisation model should help. *Actually:* each one silenced more on-screen sounds only by silencing about as many needed ones (Figure 12). Removing two wrong pictures saves $2 \times 2 = 4$ cost units; losing one needed sound costs 4: no gain. Localisation maps change little when the audio is silent or off-screen, so these models still point at a visible object [36]: they answer *where* a sound comes from, not *whether* its source is visible.

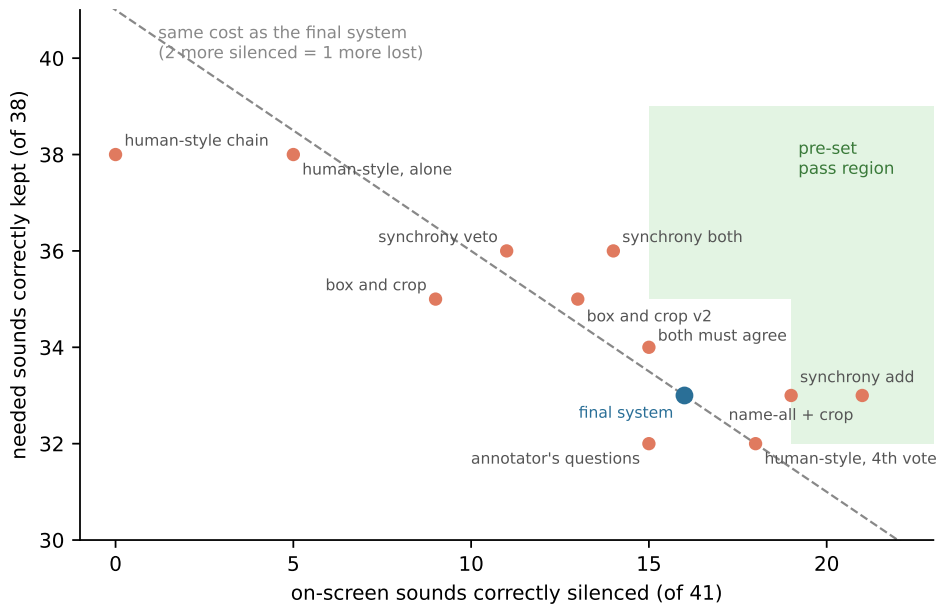


Figure 12: Every variant of the on-screen check that was tried, scored on the subset of development sounds that reach the check and whose stretches the author labelled (41 on-screen sounds, 38 needed sounds). The dashed line has the same cost as the final system: silencing two more on-screen sounds is worth losing one needed sound. All variants lie near this line, and none reached the pass region fixed before the tests. Labels name the extra tool tried: an object box, a crop around the named object, an audio-visual synchrony model, a longer human-style question chain, or a fourth vote.

7.6 Answer-form biases

Expected: a clear question gets a clear answer. *Actually:* given options (a) and (b), the 27B model picks (a) 88% of the time in one test, a known letter preference of language models [72]; asked yes/no, it leans to “no”; asked for free text, its answer can be cut off. Two workarounds are used. The scene check reads the model’s preference for “yes” over “no” as a number instead of reading its words (minus the same for the opposite question); the on-screen check asks its a/b question in both orders and abstains if the answers disagree. This removes the habits but exposes the real weakness: when the on-screen check’s own a/b question is read the same way, as a score, it separates visible from needed sounds on the labelled development stretches with an AUROC of only 0.647 (the chance that a random visible stretch scores higher than a random needed one; 0.5 is a coin flip, 1 is perfect). The habits were hiding a weakness, not causing it.

7.7 Drawing one unseen source

Expected: an audio-to-image model can draw what is heard. *Actually:* such models draw the whole soundtrack as a scene and cannot be told to leave out the dog that is already on screen. The system therefore draws from the label with a guarded text prompt – and even then a picture was named right at a glance in only 26 of 54 tries (Section 5.5). A richer prompt reached 32 of 54 but invented objects (a thud drawn as a door).

7.8 Automatic evaluation

The proposal planned an automatic score: one model describes the output video, another writes a reference of what a viewer should learn, and a third model (the judge) compares the two. *Expected:* a model judge can score the output. *Actually:* a reference written by the system’s own models rewards the system; one written by independent models agreed with the human “nothing is missing” label on 55 of 100 clips, which is chance; and the best judge scored a picture and a text tag the same (Appendix E.7). Existing labels do not help either: AudioSet names only a few sounds per clip, so on an AudioSet calibration set an earlier version of the system looked worse than showing nothing (1.650 against 0.986 per clip), although on full labels it is clearly better. Even full human labels miss some sounds (Section 8.3), and 14 borderline development sounds sit one judgement away from the other class.

7.9 Scarcity of suitable data

Clips in which an ambient sound is heard but its source is not seen are uncommon in public video. Four different sourcing routes all yielded only 15–20% usable clips, which is why the benchmark has 159 evaluation clips rather than the planned 300, and why every comparison has wide intervals.

7.10 What today’s models are missing

The failures above, and the rejected ideas of Table 14, point to six capabilities that current open models lack. Each one is a direction in which a better model, not a better rule, is needed.

1. **Hearing a quiet event under speech or music.** Taggers are trained to name the dominant sound of a clip, so a door under a conversation loses to *Speech*. Separating the speech first does not help, because the detectors have never heard separated audio (Section 7.2). What is missing is detection trained on mixtures in which the event of interest is the quiet one.
2. **Knowing when not to say yes.** A listener asked “is there a siren?” tends to agree, and a sound one listener names is often not there (Section 7.3). What is missing is a calibrated answer: a confidence that means the same thing from one question to the next, or the option to answer “not sure”.

3. **Linking an object to the sound it is making now.** Vision-language models answer “is there a bell?” when asked “is the bell ringing?” (Section 7.4). They see objects, not actions that produce sound, and they do not use the audio. What is missing is audio-visual grounding of *production*: does this object’s motion match this sound at this moment?
4. **Saying that a source is off screen.** Localisation maps change little when the audio is silent or off-screen, so these models still point at a visible object [36]. What is missing is an explicit “not in the frame” answer, which is exactly the decision this task needs.
5. **Drawing one source and leaving out the rest.** Audio-to-image models draw the whole soundtrack as one scene. What is missing is a generator that takes a negative condition from the video (“a dog, but not the dog that is already visible”) and draws one clear object for a glance.
6. **Judging per sound.** Model judges score a whole clip and cannot tell whether a picture matches a specific sound at a specific time, or whether that sound was visible. What is missing is an evaluator that works at the level of single timed sounds; until then, human per-sound labels are required.

The system in this report works around each gap with rules: two detectors, independent witnesses, a three-question vote, guarded prompts with a check-and-redraw loop, and human labels. A model that closed any one of these gaps would remove a whole family of rules.

8 Error analysis

This section looks at the remaining errors of the final system in detail, using its logged decision trail, per-item error lists and an audit of the on-screen check (sources in Appendix C).

8.1 Ceiling analysis

Figure 13 makes the stages perfect one after another; Table 6 instead makes each stage perfect on its own, leaving the rest as they are, so the two give different numbers for the same stage. The analysis needs saved pictures, so it was run on a slightly older version of the system (25 hits and 27 wrong on the development set, 55 needed sounds); the proportions, not the exact numbers, are the point. “Perfect timing” moves each picture to the labelled start of its sound; “perfect family” (a step in Figure 13) gives each picture the family of the labelled sound it overlaps. The waterfall applies the stages in the order timing, family, timing and family jointly, on-screen check, vetoes, listeners. Timing and family come first so that a picture of the right sound at the wrong moment counts as a timing error, not an on-screen error; the order changes how the gain is split between stages, not the final ceiling. Table 6 gives the order-free view.

Table 6: One stage made perfect at a time (development set; an older version of the system with 25 hits, 27 wrong and cost 2.451, scored on earlier labels with 55 needed sounds). Rows in the order of Figure 13.

Stage made perfect	Hits	Wrong	Cost
Timing	26	20	2.197
On-screen check	30	18	1.915
Vetoes and filters	30	27	2.169
Listeners and their filters	34	24	1.859
All stages perfect	43	3	0.761

The on-screen check and the listeners are the two largest losses. Even with every stage perfect, 12 of 55 needed sounds stay missed: five are heard by no model at all (a hammer, an explosion, a clang and two golf-club strikes; the last three are also families the system never draws) and seven are too faint or too badly timed.

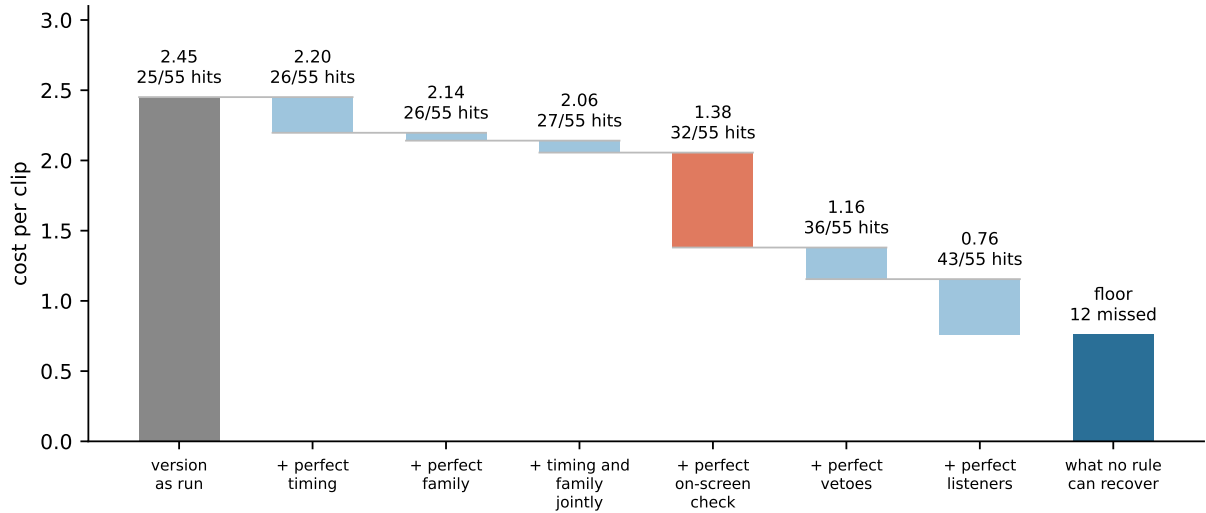


Figure 13: Ceiling analysis on a slightly older version of the system (development set, 55 needed sounds). Each bar makes one more stage perfect, using the labels, on the same detections. The 2.056 after the joint timing and family step equals the final system’s development cost only by coincidence. The largest single step is the on-screen check. Even with every stage perfect, 12 needed sounds remain missed (right): the detectors never hear them clearly enough.

8.2 The on-screen check in detail

A hand audit of the stretches the gate silenced wrongly (36 stretches on the gate-labelled clips: 16 development, 20 test; the audit tallies the same development stretches at sound level as 43 on-screen and 36 needed sounds, against the 41 and 38 of Figure 12) found two kinds of error. These are stretches, not sounds: a sound is silenced only if every one of its stretches is called visible. In 31 of the 36 the gate took *presence for production*: either the named object was in the frame but not making the sound (14, such as a church tower for a ringing bell), or another thing of the same kind was (17, such as a perched bird for a call made elsewhere). This follows from how the vote is built. Two of the three questions effectively test presence – naming an object, and asking whether that object *could* make the sound – and only the a/b question asks whether the sound is visibly happening, so the majority tests presence too. The second error is the opposite: the name question answers “nothing” on 42 % of the stretches the labels call visible or obvious, when the source is small, hidden, or its action is fine. The final system’s decision trail shows the same at sound level: of the 8 needed sounds lost at the on-screen check (5 development, 3 test), 6 were silenced although the a/b question said “not visibly happening” in every stretch; the two presence votes outvoted it.

8.3 Label strictness

In two blind checks, about 1 in 6 pictures counted as wrong was a real, wanted, off-screen sound that the labels do not list, and several others were real sounds with the source on screen. Absolute costs are therefore somewhat pessimistic. The same labels score every system, but not equally: direct audio-to-image shows more wrong pictures, so it gains more. If one in six wrong pictures of each system were not counted, the gap between the final system and direct audio-to-image would shrink by about 0.14 per clip on the test set and 0.11 on the development set (a point calculation from two blind checks), and the gap to showing nothing would grow.

8.4 Examples

Figure 14 shows real outputs of the final system, three for each outcome: hits, needed sounds missed because a similar object was in the frame, and the two main kinds of wrong picture.

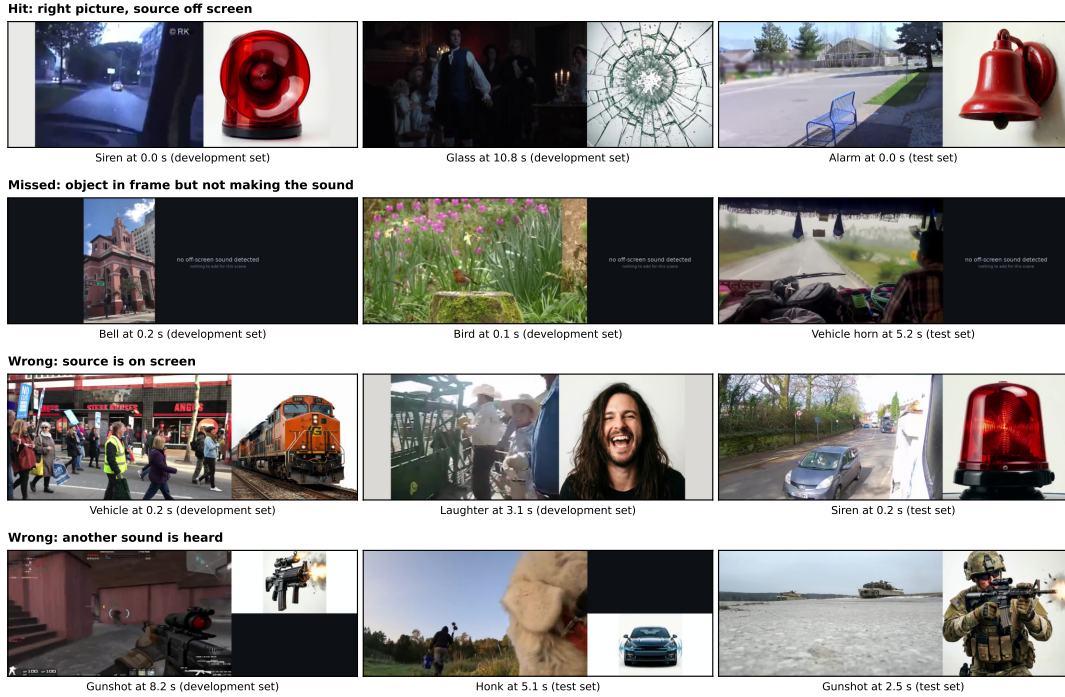


Figure 14: Outputs of the final system: each frame shows the video (left) and the picture panel (right) a moment after the sound starts; a dark slot in the panel is empty. Row 1: hits, the source is off screen. Row 2: misses; the church, the robin and the bus the camera rides in are in the frame, but none is seen making the sound that is heard (a bell, a call made by another bird, a horn), and the on-screen check silences it (the panel says no off-screen sound). Row 3: wrong pictures whose source is on screen: a truck horn at a march (detected as *Vehicle* and drawn as a train), a laughing child, an ambulance siren with the ambulance in view. Row 4: wrong pictures of a sound other than the one heard: a gunshot drawn while only footsteps are heard (the shooting starts about 2 s later), a gull’s cry detected as *Honk* and drawn as a car horn, and a gunshot drawn over the sound of a fire.

As a worked case, take a development clip filmed outside a church in Miami (clip `bell_miami`, row 2 of Figure 14). It has one needed sound: a bell from 0.2 to 14.5 s, labelled not visible. The detectors find it. The gate cuts the 14-s sound into three stretches. In each, the name question answers “church bell” (it shares the word *bell*, so this vote says visible), the a/b question says not visibly happening, and the description vote says visible. Two of three votes in every stretch: the sound is silenced and the needed bell is missed. The frames show the church; they do not show the bell ringing. The one question that asked about the action was right, and it was outvoted by two questions that measure presence.

9 Limits, threats to validity and future work

This section separates three things: what the approach itself cannot do, what the evaluation cannot show, and what could make the reported numbers wrong. Section 7 explains the first in detail.

9.1 Limits of the approach

- **Detector floor.** In the ceiling analysis of an earlier version, about one needed sound in five could not be found with today’s open detectors, mostly because it is quiet under speech or music; in the final system six needed sounds per set are heard by no detector at all. No later rule can recover them.
- **Presence is not production.** The on-screen check confuses an object being visible with the object making the sound; every alternative tried traded one error for the other.

- **No audio stage tried here separates a right off-screen name from a wrong one** well enough to add hits without two to five wrong pictures each (Section 7.3).
- **Offline only.** A 15-s clip needs about five minutes of GPU time (one NVIDIA H200) before the first picture is drawn, and each picture try takes about 20 s. The system cannot caption live television.
- **English speech only.** The speech transcript, used only to ask the vision model whether people are reacting to a sound, comes from a Whisper model [56] run for English; speech in other languages gives no useful context.

The scope of the task is set by design rather than by a limit: no model is trained, speech and music are never drawn, and one picture is shown per sound, without words (Section 1.2).

9.2 Limits of the evaluation

- **No viewer study.** The cost model assumes that a wrong picture harms a viewer half as much as a missed sound. No deaf or hard-of-hearing viewer has confirmed this, or that a generated picture is understood faster than a caption in brackets. The metric is therefore a model of the viewer, not a measurement; Figure 7 shows that, on point estimates, the main conclusion holds for any weight of a wrong picture below that of a miss.
- **One annotator.** All labels are the author’s; inter-annotator agreement is unknown.
- **Strict “wrong”.** It counts redundant and late pictures, and some “wrong” pictures are real off-screen sounds missing from the labels (Section 8.3).
- **Pictures checked by the author only.** The picture result rests on one blind glance test by the author; the automatic picture check was validated on known cases, not against a human rater.

9.3 Threats to validity

- T1. Development versus test.** Every decision was made on the development set; the test set was only scored. The test cost fell by about a third as much as the development cost (Figure 10), so the test numbers, not the development numbers, are the estimate for a new video.
- T2. Small sets.** With 71 and 88 clips, one or two pictures can decide a development comparison: one wrong picture moves the development cost by 0.028 and one hit by 0.056 per clip. The 95 % intervals of the main comparisons are 0.5 to 1.2 per clip wide.
- T3. Label changes.** The author re-checked the visibility of 14 disputed development sounds blind (3 changed; 0 of 20 random controls changed); test labels were never changed.
- T4. One baseline not run.** A video-conditioned generator raised during supervision (for example MiniMax-H3) creates new video; the per-sound score cannot evaluate it, so it was not compared.
- T5. Scored versus live system.** The scoring harness and the live pipeline differ in four small ways that do not change when a picture starts; one of them (the direction of the kinship rule, Figure 1) was not re-scored (Appendix B).

9.4 Future work

Each item addresses one of the limits above.

1. **Measure the viewer’s costs:** a study with 5–10 DHH viewers (“helped / hurt / neither” per clip) that estimates the weight of a wrong picture instead of assuming it.
2. **A second annotator** for 30 clips (the packet exists), to measure label agreement.

3. **The presence-versus-production question:** a much larger vision model, a comparison of the frame at the sound’s onset with a frame just before it, or an external box around the named object.
4. **A detector that hears sounds under speech and music,** tested on a fresh set of 422 AudioSet clips that no rule in this report has seen.

10 Conclusion

A deaf viewer misses the sounds whose source is not on screen, and no existing channel supplies them automatically. The task looks like a short chain of ready-made models, but every link fails in its own way (Section 7; the missing capabilities are listed in Section 7.10). This project built a system that works despite these failures, from open models with no training, after more than 150 full-pipeline variants (Table 5). What is new in kind is the decision itself made measurable: a per-sound benchmark that marks each sound as seen making it or not, obvious or not, and important or not, and a viewer cost that prices a miss against a wrong picture for this decision. On the test set the final system finds 37 % of the needed sounds, half of its pictures are right, and its cost is lower than all three baselines of the proposal (audio captioning scores as direct audio-to-image generation under the per-sound metric): clearly against direct audio-to-image generation ($p < 0.001$) and at the margin against showing nothing ($p = 0.044$ uncorrected, 0.087 after a Holm correction over five tests, on a test set used only for reporting). Its F1 is also higher than that of direct audio-to-image generation on the test set ($p = 0.002$; 0.007 after the same correction). The remaining errors sit where today’s open models stop: sounds the detectors barely hear under speech and music, and a vision model that takes “the object is there” for “the object is making the sound”.

Answers to the research questions.

RQ1 – which audio information matters most?

Danger sounds and key moments whose source is off screen: sirens, alarms, a baby crying, glass. They are about a quarter of the needed sounds (30 of 123), and they are the sounds the system serves best: it finds 82 % of them on the development set and 77 % on the test set, against 37 % and 27 % of ordinary events, and the on-screen check loses none of them on the test set and one on the development set, at a fraction of the baseline’s wrong pictures (Figure 6). More broadly, the information that matters is discrete, sourced events (a bell, a knock), which the benchmark’s importance rule (an event one can say in a sentence, or danger) captures; steady place noise (wind, traffic hum) is excluded by design, and it is also where detectors, gate and labels disagree most. Whether DHH viewers rank sounds the same way is untested.

RQ2 – what granularity?

Decide and score by *family* (two labels match if one lies under the other in the AudioSet tree, so *Bark* matches *Dog*, while a vague label such as *Animal* matches nothing), and draw the specific source. Matching by family was the one rule, both for merging repeated pictures and for scoring, that worked on new clips without a tuned threshold. The picture names the maker (a barking dog, not “bark”) from a hand-written table; on seven known cases only, the table was right 7 times and the vision model’s own wording 3 times.

RQ3 – can generated augmentations improve accessibility beyond subtitles?

Not shown. What is shown is that the system adds information no subtitle carries (off-screen sounds) at a modelled cost below showing nothing, and that its pictures are recognised at a glance about half the time (26 of 54, Section 5.5). A study with viewers is the necessary next step.

RQ4 – how to evaluate?

Per sound, against human labels, with the error split by cause and a cost that prices a

miss against a wrong picture, reported over a range of weights; decisions pre-registered on a development set. Automatic protocols with model-written references or model judges failed: they are circular, decide visibility at chance, or cannot tell a picture from a text tag.

RQ5 – comparison with audio-to-visual approaches?

Against the proposal’s direct audio-to-image baseline (the final system without the on-screen check, drawing every detected sound), the final system costs 0.773 less per clip on the test set ($p < 0.001$): it has two fewer hits but 38 fewer wrong pictures. Audio captioning scores the same as direct audio-to-image under the per-sound metric (a separate captioning model was not run; the captioning baseline is the same decisions shown as text); whether pictures beat text needs viewers.

Code and data availability

The code, the per-sound labels, the scoring harness, the logged decisions of the final system and the sources of this report are published as release v1.3.0 of <https://github.com/adamgavriely/MscFinalProject> [21]. The video clips are not redistributed; Appendix C explains how to reproduce every number.

References

- [1] K. Alhamoud et al. Vision-language models do not understand negation. *CVPR*, 2025. arXiv:2501.09425. Benchmark: NegBench.
- [2] O. Alonzo, H. V. Shin, D. Li. Beyond subtitles: Captioning and visualizing non-speech sounds to improve accessibility of user-generated videos. *ASSETS*, 2022. doi:10.1145/3517428.3544808.
- [3] AndreasXi. FineLAP, model card. <https://huggingface.co/AndreasXi/FineLAP>, accessed 3 October 2026.
- [4] S. Bai et al. (Qwen Team). Qwen2.5-VL technical report. arXiv:2502.13923, 2025.
- [5] Ç. Bilen, G. Ferroni, F. Tuveri, J. Azcarreta, S. Krstulović. A framework for the robust evaluation of sound event detection. *ICASSP*, 2020. arXiv:1910.08440.
- [6] B. C. Biner, F. M. Sofian, U. B. Karakaş, D. Ceylan, E. Erdem, A. Erdem. SonicDiffusion: Audio-driven image generation and editing with pretrained diffusion models. arXiv:2405.00878, 2024.
- [7] Black Forest Labs. FLUX.1-schnell, model card. <https://huggingface.co/black-forest-labs/FLUX.1-schnell>, accessed 3 October 2026.
- [8] P. Cai, Y. Song, Q. Gu, N. Jiang, H. Song, I. McLoughlin. Detect Any Sound: Open-vocabulary sound event detection with multi-modal queries. *ACM MM*, 2025. arXiv:2507.16343.
- [9] H. Chen, W. Xie, A. Vedaldi, A. Zisserman. VGGSound: A large-scale audio-visual dataset. *ICASSP*, 2020. arXiv:2004.14368.
- [10] S. Chen et al. BEATs: Audio pre-training with acoustic tokenizers. *ICML*, 2023. arXiv:2212.09058.
- [11] W. Chen, Y. Liang, Z. Ma, Z. Zheng, X. Chen. EAT: Self-supervised pre-training with efficient audio transformer. *IJCAI*, 2024. doi:10.24963/ijcai.2024/421. arXiv:2401.03497.
- [12] Y. Chu et al. Qwen2-Audio technical report. arXiv:2407.10759, 2024.
- [13] C. de Lacerda Pataca, S. Hassan, N. Tinker, R. L. Peiris, M. Huenerfauth. Caption Royale: Exploring the design space of affective captions from the perspective of deaf and hard-of-hearing individuals. *CHI*, 2024. doi:10.1145/3613904.3642258.
- [14] C. de Lacerda Pataca, M. Watkins, R. Peiris, S. Lee, M. Huenerfauth. Visualization of speech prosody and emotion in captions: Accessibility for deaf and hard-of-hearing users. *CHI*, 2023. doi:10.1145/3544548.3581511.
- [15] Described and Captioned Media Program. Captioning Key: Sound effects and music. <https://dcmp.org/learn/captioningkey>, accessed 27 September 2026.

- [16] H. Dinkel, Z. Yan, Y. Wang, J. Zhang, Y. Wang, B. Wang. Scaling up masked audio encoder learning for general audio classification (Dasheng). *Interspeech*, 2024. arXiv:2406.06992.
- [17] K. Drossos, S. Adavanne, T. Virtanen. Automated audio captioning with recurrent neural networks. *WASPAA*, 2017. arXiv:1706.10006.
- [18] K. Drossos, S. Lipping, T. Virtanen. Clotho: An audio captioning dataset. *ICASSP*, 2020. arXiv:1910.09387.
- [19] C. Drummond, R. C. Holte. Cost curves: An improved method for visualizing classifier performance. *Machine Learning* 65(1):95–130, 2006.
- [20] B. Elizalde, S. Deshmukh, M. Al Ismail, H. Wang. CLAP: Learning audio concepts from natural language supervision. *ICASSP*, 2023. arXiv:2206.04769.
- [21] A. Gavriely. Visual Augmentation of Audio Semantics for Accessibility: code, labels and report, release v1.3.0. GitHub, 2026. <https://github.com/adamgavriely/MscFinalProject/releases/tag/v1.3.0>.
- [22] D. Ge, S. Liu, C. Gong, X.-L. Zhang, C. Zhang, X. Li. Towards expressive and faithful audio-to-image generation: A unified multimodal dataset and synthesis framework. *ACM MM*, 2026. arXiv:2608.09529. System: AudioCanvas.
- [23] J. F. Gemmeke et al. Audio Set: An ontology and human-labeled dataset for audio events. *ICASSP*, 2017. doi:10.1109/ICASSP.2017.7952261.
- [24] T. Geng, T. Wang, J. Duan, R. Cong, F. Zheng. Dense-localizing audio-visual events in untrimmed videos: A large-scale benchmark and baseline. *CVPR*, 2023. arXiv:2303.12930.
- [25] Y. Gong, Y.-A. Chung, J. Glass. AST: Audio spectrogram transformer. *Interspeech*, 2021. arXiv:2104.01778.
- [26] Google. Gemma-4-31B-it, model card. <https://huggingface.co/google/gemma-4-31B-it>, accessed 3 October 2026.
- [27] J. Hai, H. Wang, W. Guo, M. Elhilali. FlexSED: Towards open-vocabulary sound event detection. *WASPAA*, 2025. arXiv:2509.18606.
- [28] V. Iashin, W. Xie, E. Rahtu, A. Zisserman. Synchformer: Efficient synchronization from sparse cues. *ICASSP*, 2024. arXiv:2401.16423.
- [29] JaidedAI. EasyOCR. <https://github.com/JaidedAI/EasyOCR>, accessed 3 October 2026.
- [30] D. Jain et al. Head-mounted display visualizations to support sound awareness for the deaf and hard of hearing. *CHI*, 2015. doi:10.1145/2702123.2702393.
- [31] D. Jain et al. Exploring sound awareness in the home for people who are deaf or hard of hearing. *CHI*, 2019. doi:10.1145/3290605.3300324.
- [32] D. Jain et al. HomeSound: An iterative field deployment of an in-home sound awareness system for deaf or hard of hearing users. *CHI*, 2020. doi:10.1145/3313831.3376758.
- [33] D. Jain, H. Ngo, P. Patel, S. Goodman, L. Findlater, J. Froehlich. SoundWatch: Exploring smartwatch-based deep learning approaches to support sound awareness for deaf and hard of hearing users. *ASSETS*, 2020. doi:10.1145/3373625.3416991.
- [34] D. Jain et al. ProtoSound: A personalized and scalable sound recognition system for deaf and hard-of-hearing users. *CHI*, 2022. arXiv:2202.11134.
- [35] A. Q. Jiang et al. Mistral 7B. arXiv:2310.06825, 2023.
- [36] X. Juanola, G. Haro, M. Fuentes. A critical assessment of visual sound source localization models including negative audio. *ICASSP*, 2025. arXiv:2410.01020.
- [37] C. D. Kim, B. Kim, H. Lee, G. Kim. AudioCaps: Generating captions for audios in the wild. *NAACL*, 2019. doi:10.18653/v1/N19-1011.
- [38] Kimi Team. Kimi-Audio technical report. arXiv:2504.18425, 2025.
- [39] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, M. D. Plumbley. PANNs: Large-scale pre-trained audio neural networks for audio pattern recognition. *IEEE/ACM TASLP* 28:2880–2894, 2020. doi:10.1109/TASLP.2020.3030497. arXiv:1912.10211.
- [40] H. Liu, C. Li, Q. Wu, Y. J. Lee. Visual instruction tuning. *NeurIPS*, 2023. arXiv:2304.08485.

- [41] Loie. SpotSound (SpotSound-A adapter for Audio Flamingo 3), model card. <https://huggingface.co/Loie/SpotSound>, accessed 3 October 2026.
- [42] L. May et al. Unspoken sound: Identifying trends in non-speech audio captioning on YouTube. *CHI*, 2024. doi:10.1145/3613904.3642162.
- [43] L. May et al. “Choices? That’s the dream”: Challenges and opportunities in non-speech information closed-captioning. *Frontiers in Computer Science* 7, 2025. doi:10.3389/fcomp.2025.1575176.
- [44] A. Mesaros, T. Heittola, T. Virtanen. Metrics for polyphonic sound event detection. *Applied Sciences* 6(6):162, 2016.
- [45] Meta. SAM 3, model card. <https://huggingface.co/facebook/sam3>, accessed 3 October 2026.
- [46] Meta. SAM-Audio (large), model card. <https://huggingface.co/facebook/sam-audio-large>, accessed 3 October 2026.
- [47] M. Minderer, A. Gritsenko, N. Houlsby. Scaling open-vocabulary object detection. *NeurIPS*, 2023. arXiv:2306.09683.
- [48] NVIDIA. Audio Flamingo Next, model card. <https://huggingface.co/nvidia/audio-flamingo-next-hf>, accessed 3 October 2026.
- [49] OpenMOSS Team. MOSS-Audio-8B-Thinking, model card. <https://huggingface.co/OpenMOSS-Team/MOSS-Audio-8B-Thinking>, accessed 3 October 2026.
- [50] A. Owens, A. A. Efros. Audio-visual scene analysis with self-supervised multisensory features. *ECCV*, 2018. arXiv:1804.03641.
- [51] PixArt-alpha. PixArt- Σ XL-2 1024-MS, model card. <https://huggingface.co/PixArt-alpha/PixArt-Sigma-XL-2-1024-MS>, accessed 3 October 2026.
- [52] D. Podell et al. SDXL: Improving latent diffusion models for high-resolution image synthesis. *ICLR*, 2024. arXiv:2307.01952.
- [53] Qwen Team. Qwen3.8-27B, model card. <https://huggingface.co/Qwen/Qwen3.8-27B>, accessed 3 October 2026.
- [54] Qwen Team. Qwen-Image-2512, model card. <https://huggingface.co/Qwen/Qwen-Image-2512>, accessed 3 October 2026.
- [55] A. Radford et al. Learning transferable visual models from natural language supervision. *ICML*, 2021. arXiv:2103.00020.
- [56] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, I. Sutskever. Robust speech recognition via large-scale weak supervision. *ICML*, 2023. arXiv:2212.04356.
- [57] P. Revuelta, T. Ortiz, M. J. Lucía, B. Ruiz, J. M. Sánchez-Pena. Limitations of standard accessible captioning of sounds and music for deaf and hard of hearing people: An EEG study. *Frontiers in Integrative Neuroscience* 14, 2020. doi:10.3389/fnint.2020.00001.
- [58] S. Rouard, F. Massa, A. Défossez. Hybrid transformers for music source separation. *ICASSP*, 2023. arXiv:2211.08553. Model: HT-Demucs.
- [59] A. Senocak, H. Ryu, J. Kim, T.-H. Oh, H. Pfister, J. S. Chung. Sound source localization is all about cross-modal alignment. *ICCV*, 2023. arXiv:2309.10724.
- [60] Stability AI. SDXL-Turbo, model card. <https://huggingface.co/stabilityai/sdxl-turbo>, accessed 3 October 2026.
- [61] StepFun Audio Team. Step-Audio 2 technical report. arXiv:2507.16632, 2025. Model used: Step-Audio-2-mini.
- [62] K. Sung-Bin, A. Senocak, H. Ha, A. Owens, T.-H. Oh. Sound to visual scene generation by audio-to-visual latent alignment. *CVPR*, 2023. arXiv:2303.17490.
- [63] SYSTRAN. faster-whisper. <https://github.com/SYSTRAN/faster-whisper>, accessed 3 October 2026.
- [64] Y. Tian, D. Li, C. Xu. Unified multisensory perception: Weakly-supervised audio-visual video parsing. *ECCV*, 2020. arXiv:2007.10558. Dataset: LLP.
- [65] E. Tzinis et al. Into the wild with AudioScope: Unsupervised audio-visual separation of on-screen sounds. *ICLR*, 2021. arXiv:2011.01143.

- [66] S. J. Um, D. Kim, S. Lee, J. U. Kim. Object-aware sound source localization via audio-visual scene understanding. *CVPR*, 2025. arXiv:2506.18557.
- [67] C. Wu et al. Qwen-Image technical report. arXiv:2508.02324, 2025.
- [68] Y. Wu et al. FLAM: Frame-wise language-audio modeling. *ICML*, 2025. arXiv:2505.05335.
- [69] J. Xu et al. Qwen3-Omni technical report. arXiv:2509.17765, 2025.
- [70] xavijuanola. SSL_SaN, code and weights (BMVC 2025). https://github.com/xavijuanola/SSL_SaN, accessed 3 October 2026.
- [71] X. Zhai, B. Mustafa, A. Kolesnikov, L. Beyer. Sigmoid loss for language image pre-training. *ICCV*, 2023. arXiv:2303.15343.
- [72] C. Zheng, H. Zhou, F. Meng, J. Zhou, M. Huang. Large language models are not robust multiple choice selectors. *ICLR*, 2024. arXiv:2309.03882.
- [73] L. Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*, 2023. arXiv:2306.05685.

A Prompts used in the system

All questions are asked with greedy decoding and, for Qwen3.8-27B, thinking off. $\langle \text{label} \rangle$ is the family name, lower-cased and cut before a comma or bracket. Source: `src/stage5_cross_modal_analysis/reason.py`, `benchmark/listener_round.py`, `benchmark/gold/listener_variants.py`, `src/stage6_visual_augmentation/group.py`, `depict.py`, `verify.py`, `gen_screen.py`.

Gate: name vote

“These frames are from the moment a sound of $\langle \text{label} \rangle$ was heard. Name the thing in these frames that is making that sound. Answer with a short noun phrase of at most 5 words. If nothing that could make that sound is visible in these frames, answer exactly: nothing.”

Gate: follow-up (text only)

“Sound heard: $\langle \text{label} \rangle$ / Thing visible in the video: $\langle \text{named} \rangle$ / Could that thing be what is making that sound? Answer yes or no.”

Gate: a/b vote (asked in both orders)

“These frames are from the moment a sound of $\langle \text{label} \rangle$ was heard. Judge from the frames alone. (a) you can SEE $\langle \text{label} \rangle$ happening on screen – the source is in the frames and visibly making that sound (b) $\langle \text{label} \rangle$ is not visibly happening in these frames. Answer with the letter only.”

Gate: describe vote

“Describe what is happening in these frames in one sentence. Name what you see and what it is doing. No more than 25 words.”

Speech context (text only, both orders)

“... Are they (a) reacting to that sound or talking about it, or (b) not referring to that sound?”

Scene check (witness rule; logit read)

“Could the sound of $\langle \text{label} \rangle$ plausibly be heard in this scene? Answer yes or no.” and the twin “Could the sound of $\langle \text{label} \rangle$ NOT plausibly be heard in this scene? Answer yes or no.”

Listener yes/no (Qwen3-Omni, Audio Flamingo Next)

“Is the sound of $\langle \text{label} \rangle$ present in this recording? Answer yes or no.”

Listener inventory

“List every distinct non-speech sound you hear in this recording, one per line, most prominent first.” (answer limited to 64 tokens, about 50 words)

Grouping (Qwen3-Omni, both orders)

“You hear a $\langle \text{label} \rangle$ sound near the start and again near the end of this recording. Is the $\langle \text{label} \rangle$ near the end $\langle o_1 \rangle$, or $\langle o_2 \rangle$? Answer with exactly one word: $\langle w_1 \rangle$ or $\langle w_2 \rangle$.” with the options “the same continuing sound as at the start (for example one alarm or engine with a short pause)” and “a new, separate event (for example a second bark, knock or shot)”.

Depict-event

“Is “ $\langle \text{event} \rangle$ ” visibly happening in these frames? Answer yes or no.” and “Do these frames show no “ $\langle \text{event} \rangle$ ” happening? ...”; then (text only) “Could the sound of $\langle B \rangle$ be mistaken for the sound of $\langle A \rangle$?” in both directions.

Picture check

“Look at this picture. What is the main thing shown in it? Choose exactly one option. ... Answer with the letter only.”

Picture prompt tail

“, the whole thing fully in frame, caught at the moment it makes the sound with its visible effect, one large subject filling the picture, plain white background”; negative prompt: scenery, landscape, background scene, room interior, street, buildings, sky background, text, letters, watermark.

B System details and configuration

This appendix gives the parts of the final system that the main text summarises: every step of the detection stage, the label filter, how the scored numbers relate to the live pipeline, the exact definition of the direct audio-to-image baseline, and every configuration value.

B.1 Detection (stage 4) in full

Stage 4 is where most of the development effort went. It turns the audio into a list of timed candidate sounds, each with a family label and a confidence. The entry point is `detect_events` (`src/stage4_audio_event_detection/__init__.py`); unions and vetoes are in `fuse_flexsed`, the filters on rescued spans in `filter_rescued`.

Models

BEATs

iter3+ fine-tuned on AudioSet-2M (checkpoint `BEATs_iter3_plus_AS2M_finetuned_on_AS2M_cpt2`). It scores a whole window, so it is run on 2-s windows every 0.25 s; the first two seconds are mirror-padded so a sound at the very start can be heard; each window’s score is stamped 0.5 s before the window ends, so a detected start is never earlier than the true one.

FlexSED

an open-vocabulary, frame-level detector [27]: a self-supervised audio encoder plus a CLAP text encoder (`laion/clap-htsat-unfused`), trained on AudioSet-Strong. It is asked one plain query per family over the 215 *depictable* families (`benchmark/gold/depictable_vocab.json`), 25 frames per second. It hears quiet sounds under speech and music far better than BEATs.

PANNs CNN14

(sound-event version, 32 kHz) used only as a clip-level veto.

DASM

Detect Any Sound Model [8] (ACM MM 2025; weights `CPF2/detect_any_sound`), queried with “sound of ⟨family⟩” over the same 215 families, 50 frames per second. Used as a witness and a vote.

Listeners

`Qwen/Qwen3-Omni-30B-A3B-Instruct` [69] and `nvidia/audio-flamingo-next-hf` [48]. Each is asked a yes/no question (“Is the sound of ⟨family⟩ present in this recording?”), scored as the logit of *yes* minus the logit of *no* and an open inventory (“List every distinct non-speech sound you hear in this recording, one per line, most prominent first.”). An inventory line names a family if it matches its name, an AudioSet synonym, a parent or a child, or else has sentence-embedding cosine > 0.6 (`all-mpnet-base-v2`). The audio cut is the candidate span ± 1 s, at least 1 s long.

FineLAP

(`AndreasXi/FineLAP`, MIT) [3]: a frame-level audio-text model used as a last veto on rescued sounds.

Steps in execution order Table 7 gives the 16 steps. Two words are used throughout: a *rescued* span is one added by a listener after the detectors missed it; a *twinned* span is one both detectors raised.

Table 7: Stage 4 of the final system, step by step.

#	Step	Rule	Variable(s)
1	BEATs spans	a span of any class where the window score ≥ 0.175 ; kept if ≥ 0.3 s long; confidence = peak	AED_THRESHOLD 0.175, AED_MIN_DUR 0.3
2	FlexSED spans	same extractor on FlexSED frames at bar 0.8, ≥ 0.3 s	FLEXSED_BAR 0.8
3	twin union	a FlexSED span with a same-family BEATs span within 1 s is merged into it (earlier start kept; confidence = the stronger side, rescaled); otherwise kept as FlexSED-only	TWIN_MAX, UNION_START min
4	mirror veto + listener keep	a BEATs-only span is dropped if FlexSED’s top query in its window is another family ≥ 0.7 while its own family is < 0.4 ; put back if the Qwen listener accepts it and an inventory names it	MIRROR_VETO 0.7, KEEP_NEEDS_V4
5	masked-weak veto	a BEATs-only span with confidence < 0.5 under speech or music (≥ 0.3) is dropped unless a listener accepts it	MASKED_WEAK_VETO
6	DASM clip veto	a span whose family DASM never reaches 0.0839 anywhere in the clip is dropped unless a listener keeps it	DASM_CLIP_VETO
7	witness rule	pass if DASM ≥ 0.35 for the family within the span ± 0.5 s; else pass if <i>both</i> listener inventories name the family; if exactly one does, ask the scene check (Section B.1); otherwise drop	DASM_LOCAL_VETO 0.35, DASM_LOCAL_KEEP both, SCENE_FIT_LOGIT
8	inventory keep	a span with confidence ≥ 0.35 for which both inventories exist and neither names its family (or a descendant) is dropped, unless DASM ≥ 0.575 within ± 0.5 s	KEEP_NEEDS_V4_ALL on, KEEP_NEEDS_V4_ALL_DASM_KEEP
9	band-twin pull	the start moves back to an earlier same-family FlexSED run (≥ 0.5) that ends ≤ 1 s before and starts ≤ 1.5 s before it	BAND_TWIN_PULL 0.5
10	continuation veto	a span is dropped when a same-family FlexSED run (≥ 0.5) began ≥ 1.5 s earlier and is still on at its start (it is a later piece of a sound already shown)	CONTINUATION_VETO 0.5
11	FlexSED veto	a span whose family FlexSED never reaches 0.3 in the clip is dropped	FLEXSED_VETO 0.3
12	PANNs veto	a FlexSED-only span whose family PANNs never reaches 0.05 is dropped unless a listener accepts it	PANNS_VETO 0.05
13	listener band rescue	a FlexSED run with peak in $[0.5, 0.8)$ and no same-family span is added if accepted: peak ≥ 0.6 needs the Qwen inventory; < 0.6 needs both inventories	LISTENER_RESCUE, LISTENER_RULE TIER
14	DASM rescue	a DASM run (≥ 0.575) with no detector span nearby is added if both inventories name the family and the family is new in the clip	DASM_RESCUE, DASM_RESCUE_NEW_ONLY
15	onset refinement	for BEATs spans: silence the first t seconds of the first firing window for 26 values of t and take the first t at which 10 % of the class evidence is gone; never earlier than the detected start	ONSET_CAM, ONSET_MONOTONE
16	rescued-span filters	on rescued spans only: FineLAP ≥ 0.329 ; DASM ≥ 0.575 within ± 0.5 s; at most one rescued span per family per clip (the earliest)	FINELAP_VETO 0.329, LISTENER_DASM_VOTE, LISTENER_ONCE

The DASM thresholds 0.0839 and 0.575 and the FineLAP bar 0.329 were calibrated during development (Appendix E); every other threshold was set once and not tuned on the test set. Steps 4–8 and 12–16 depend on listener answers that are computed from a first pass with an earlier configuration (`use_scored()`; the saved lists of candidate sounds, called pools), so the final detector cannot run without that pass (Section C.1).

The scene check When exactly one listener names a weak sound, the witness rule asks the vision model whether the sound is plausible in the scene. The span is cut into stretches of about 5 s; for each, six frames from 1 s before to 1 s after are shown with the question “Could the sound of $\langle \text{label} \rangle$ plausibly be heard in this scene? Answer yes or no.” and its negated twin. The answer is read from the first output token as $d = m(Q) - m(\bar{Q})$, where $m = \max$ logit over {yes, Yes} minus $\max$ logit over {no, No}; $d > 0$ counts as yes, and the sound passes if most stretches say yes. Reading the logits replaced an earlier text answer that was cut off after four tokens on 10–15 % of calls (Appendix E).

B.2 What may be drawn

The label filter (`LABEL_FILTER="depictable"`, `src/labels.py`, `is_salient_nonspeech`) maps every label to its family and removes: speech with words and its descendants (Speech, Conversation, Narration, Hubbub, Whispering, ...), music and singing and their descendants, the “channel, environment and background” branch, silence and generic labels, textures (Breathing, Rumble, Hum, Whir, Rustle, Wind and its descendants), and labels with no clear source (Sound effect, Onomatopoeia, generic impacts). Everything else is drawable, including non-word vocal events (shout, scream, laughter, cough, snoring). FlexSED and DASM are asked only about the 215 drawable families.

B.3 Scoring harness versus live pipeline

The numbers in this report come from the scoring harness, which runs the pipeline’s own stage-4 functions on cached model scores and replays the gate answers of a full run. A parity check (`benchmark/gold/parity_check.py`) and an independent re-export of every decision (the Decision Inspector, Section C.1) reproduce the published counts exactly (0 of 159 clips differ). The live configuration `use_shipped()` differs from the scored one in these known ways:

- **Kinship direction.** Live: a visible label silences only the same or a more specific label. Scored: either direction. This was a deliberate logic fix made after the scored tables; its effect on the final numbers was not re-measured.
- **Picture end.** Live pictures end at most 1 s after their sound; scored pictures had no cap. Starts – the scored quantity – are identical; on 3 test clips a picture ends 0.04–0.38 s earlier live.
- **Subject path.** `main.py` runs planning, the on-screen check and the duplicate check with the scored settings and switches to the final picture settings only for drawing, so the pictures it shows are designed to match the scored run.
- **Pictures.** Scored runs used placeholder or FLUX.1-schnell pictures; the live system uses Qwen-Image with the check. Pictures are not part of the per-sound score.

B.4 The direct audio-to-image baseline

The baseline of Section 6.1 is the final system with the on-screen check switched off. The steps that need the vision model’s answers about the frames are therefore off too: the transcript question (which may rescue a marginal sound), the frame-based choice between two labels on one event, and the depict-event check. In the final runs on both sets the transcript rescue never fired and the label choice changed nothing, so in practice the baseline differs from the final system in the on-screen

check and the depict-event check. One frame-reading step stays, the witness rule’s scene question (Section B.1), because it belongs to detection, not to the on-screen check. The detection stack, listeners, display rules and picture model are the same, and every surviving sound is drawn.

B.5 Configuration values

Values set by `config.use_shipped()` (`config.py`), read by running it. “ $\rightarrow$ ” gives the module default and the shipped value.

Table 8: Configuration values of the final system.

Variable	Value	Meaning
AED_THRESHOLD	0.175	BEATs extraction bar
AED_MIN_DUR	0.5 $\rightarrow$ 0.3	shortest span (s)
DISPLAY_THRESHOLD	0.35	display bar (stage 5)
FLEXSED_BAR	0.8	FlexSED extraction bar
FLEXSED_VETO	0 $\rightarrow$ 0.3	FlexSED clip veto
PANNS_VETO	0.05	PANNS clip veto on FlexSED-only spans
TWIN_MAX	True	twin confidence = stronger side
MIRROR_VETO	0.7	mirror veto bar
LISTENER_CONFIRMED_MIRROR	True	listener can undo the mirror veto
KEEP_NEEDS_V4	True	a keep must also be named in an inventory
MASKED_WEAK_VETO, _AF	True	masked-weak veto
DASM_CLIP_VETO	0.0839	DASM clip veto
DASM_LOCAL_VETO	0.35	witness rule bar
DASM_LOCAL_KEEP	both	both listeners can witness
SCENE_FIT_LOGIT	True	scene check by logit margin
KEEP_NEEDS_V4_ALL	onto	inventory keep incl. descendants
KEEP_NEEDS_V4_ALL_DASM_KEEP	True	DASM can keep it
BAND_TWIN_PULL	0.5	band-twin pull
CONTINUATION_VETO	0.5	continuation veto
LISTENER_RESCUE	True	band rescue
LISTENER_LO	0.4 $\rightarrow$ 0.5	lowest FlexSED peak rescued
LISTENER_RULE	TIER	Qwen alone ≥ 0.6 , both below
DASM_RESCUE, DASM_RESCUE_NEW_ONLY	True	DASM rescue of new families
FINE LAP_VETO	0.329	FineLAP veto on rescued spans
LISTENER_DASM_VOTE / _BAR	True / 0.575	DASM vote on rescued spans
LISTENER_ONCE	True	one rescue per family per clip
ONSET_CAM, ONSET_MONOTONE	True	occlusion onsets, never earlier
MAX_SPAN	8.0 $\rightarrow$ None	no length cap
LABEL_FILTER	depictable	what may be drawn
VLM_MODEL	Qwen/Qwen3.8-27B	gate, subject, checks
VLM_THINKING	False	thinking off
VISIBILITY_STRETCH / _RULE	5.0 / majority	gate stretches and vote
KINSHIP_DIRECTED	True (scored: False)	kinship direction
VIDEO_BACKEND	owlv2	stage-2 objects
DEPICT_EVENT	True	depict-event drop
MERGE_GAP	2.0 $\rightarrow$ 2.5	repeat merge (s)
MIN_DWELL	1.5	minimum on screen (s)
MAX_AFTER_END	1.0 (scored: None)	cap past the real end (s)

Variable	Value	Meaning
GROUP_ASK / GROUP_MAX_GAP	True / 8.0	grouping by Qwen3-Omni
MAX_SLOTS	3	simultaneous pictures
GEN_MODEL	Qwen/Qwen-Image-2512	picture model (FLUX retired)
RESOLUTION	1024×1024	picture size
PICTURE_V3, _SCENE, _FINAL, _MAKER	True	subject writing
PICTURE_VERIFY / _TRIES	True / 5	check and redraw
CONFIDENCE_FADE	False	full opacity

C Reproducibility and sources of the numbers

C.1 Reproduction

The development record (pre-registrations, daily notes, reviews, the labelling tool) is not part of the current release, which holds only the code, the labels, the scoring and this report. It is kept in full in release v1.2.0 (<https://github.com/adamgavriely/MscFinalProject/releases/tag/v1.2.0>); every path below that starts with docs/history/, docs/README.md or tagger/ refers to that release.

Code. Release v1.3.0 of the repository. The final system is the configuration `config.use_shipped()` in `config.py`; in the scoring harness it is the variant SHIP8+MD3+WW5+SL (Appendix D).

Environment. Python 3.11 with PyTorch 2.5.1, transformers 5.16.1, diffusers, faster-whisper, panns-inference, easyocr and sentence-transformers. FineLAP needs transformers 4.51.3 and runs in a second environment, called as a subprocess (variable `FINE LAP_PYTHON`). FlexSED and DASM are imported from their own repositories, located through the environment variables `FLEXSED_ROOT`, `T4SED_ROOT` and `BERT_DIR` (see the README); the DASM weights are loaded from the local DASM repository. Models (Hugging Face identifiers): Qwen/Qwen3.8-27B, Qwen/Qwen3-Omni-30B-A3B-Instruct, nvidia/audio-flamingo-next-hf, AndreasXi/FineLAP (revision b419aa2), CPF2/detect_any_sound, Qwen/Qwen-Image-2512, google/owlv2-base-patch16-ensemble, BEATs iter3+ AS2M cpt2 and PANNs CNN14. Revisions other than FineLAP are not pinned.

Hardware and running time. One GPU with about 80 GB of memory (an NVIDIA H200 was used; two A100 80 GB cards also work). Qwen-Image needs about 57 GB in bf16, and the picture check needs the picture model and the vision model loaded together. Preparing the listener inputs takes about five minutes per clip; each picture try takes about 20 s.

Running the system on one video.

```
python main.py --input clip.mp4 --device cuda
```

This writes `data/output/<clip>_augmented.mp4` (the video with the picture panel) and the intermediate files under `data/work/<clip>/. The run first computes the per-clip model inputs the detector needs (FlexSED, BEATs and PANNs scores, the listener answers, DASM and FineLAP scores), then runs detection, the on-screen check and the pictures. Detection and the on-screen check run with exactly the settings of the scored results; only the picture step uses the final picture settings. A folder of clips is processed by calling main.py once per clip.`

Reproducing the reported numbers. The scoring harness reads the gold labels and the cached model outputs of each clip and replays the pipeline’s own decision functions:

```
# Table 3: final system and baselines
python benchmark/gold/final_vs_baselines.py
# configuration check
python benchmark/gold/parity_check.py SHIP8+MD3+WW5+SL
python benchmark/gold/inspector_trail_export.py \
    --expect DEV=29/58/15/6/7/2/2.056 --expect TEST=24/65/24/4/15/5/2.409
# figures of the report
python docs/report/figures/make_results_figure.py
python docs/report/figures/make_beta_figure.py
python docs/report/figures/make_body_figures.py
python docs/report/figures/make_analysis_figures.py
```

`python docs/report/figures/make_examples_figure.py <folder with media/bysig>`

The last command re-runs the final system with every decision logged and stops unless the counts match exactly; it reproduces all 159 clips. The figure scripts read the result files above and `docs/inspector2/data.js`; the example figure also needs the rendered output videos, which are not in the repository. The cached model outputs of the benchmark clips are not part of the repository (they are large); they are rebuilt from the clips with `benchmark/gold/tagger_prep.py`.

Decision trail. `docs/inspector2/` is a static page over `data.js` (159 clips) that shows, for every labelled sound and every candidate of the final system, each step it passed or failed, the value, the threshold and the model answers. The logging (`src/trail.py`) only records and never changes a decision.

Where the records are. The development record is in `docs/history/`: pre-registrations with their results (`docs/history/preregistrations/`), review panels, daily notes and analyses; `docs/README.md` gives the map. The gold labels are `benchmark/gold/annotations/gold_AG.json`.

C.2 Sources of the numbers

Every number in the main body comes from one of the files below (paths relative to the repository root). Counts of the final system are also reproduced exactly by the decision-trail export (`docs/inspector2/data_parity.json`, "exact": true).

The configuration of Appendix B is `config.py` (`use_shipped()`); the development history of Appendix E is in `docs/history/` (pre-registrations, panels, daily notes, reviews).

Table 9: Source file of every number in the report.

Numbers	File
Table 1 (set sizes, needed sounds)	<code>benchmark/gold/annotations/gold_AG.json</code> , counted with <code>benchmark/gold/score_per_sound.py</code>
labelling rules (Section 3.2)	<code>benchmark/gold/tool_template.html</code> , <code>tagger/index.html</code> , <code>docs/history/plans/metric_per_sound.md</code>
scoring rules, cost (Section 3)	<code>benchmark/gold/score_per_sound.py</code>
bootstrap and pass rules	<code>benchmark/gold/dev_candidates_check.py</code> ; <code>benchmark/gold/holm_table.py</code>
F1 intervals, Figure 6 (danger sounds)	<code>benchmark/gold/final_vs_baselines_extra.json</code> , from <code>benchmark/gold/final_vs_baselines_extra.py</code>
final system and baselines, Table 3	<code>benchmark/gold/final_vs_baselines.json</code> ; <code>docs/history/review/detector_final_2026-10-02.md</code> ; <code>docs/inspector2/data_parity.json</code>
Holm correction over five tests (Section 6.3)	the five p -values of <code>benchmark/gold/final_vs_baselines.json</code> and <code>benchmark/gold/final_vs_baselines_extra.json</code>
intervals at each β (Section 6.4)	per-clip hits and wrong pictures of <code>docs/inspector2/data.js</code> , with the bootstrap of Section 3.5
on-screen weight (Section 6.4)	derived from Table 3; earlier versions in <code>benchmark/gold/visible_weight_sweep.md</code>
fate of misses (Section 6.5, Figures 8 and 9; the 8 needed sounds lost at the on-screen check, Section 8.2)	<code>docs/inspector2/data.js</code> (field <code>lost_at</code>), grouped into bars by <code>LOST_AT_BAR</code> in <code>docs/report/figures/make_analysis_figures.py</code> ; <code>docs/history/review/inspector_trail_ready.md</code>
Table 6 (ceiling)	<code>benchmark/gold/ceiling_ship7.md</code>
September gate comparison (Appendix E)	<code>benchmark/gold/holm_test_final_v33_test_bench.json</code>
gate accuracy, failure modes (31 of 36 wrong silences: 14 of class i, 17 of class ii; 42 % “nothing”)	<code>docs/history/review/vlm_panel_2026-10-01/f1_audit.md</code> , sections 4–7

Numbers	File
AudioSet calibration set (1.650 against 0.986)	docs/history/preregistrations/prereg_round12_v2.md
AUROC 0.647 of the a/b question read as a score	benchmark/gold/logit_gate_gold.json; round 62 in docs/history/preregistrations/prereg_round13_detector_push.md
listener agreement on the screening set	docs/history/review/improvements_log_2026-09-30.md; docs/history/preregistrations/prereg_round13_detector_push.md
onset timing of an early version (file name historical)	benchmark/eval_dcase_onset.json
visibility check of an early version (file name historical)	benchmark/eval_dcase_visibility.json
picture glance test	docs/history/preregistrations/picture_v3_prereg.md
speech removal (9.5 % to 4.8 %)	docs/history/daily_notes/LEDGER_2026-09-26.md; docs/history/panels/fable_briefs/2026-09-16_ten_fable_panel_addendum.md
earlier label agreement (78 %, $\kappa = 0.60$)	docs/history/daily_notes/HANDOFF_2026-09-08.md; docs/history/earlier_drafts/thesis/ch4_benchmark.md
gate balanced accuracy 0.61 to 0.62	docs/history/analyses/GOLD_RERUN_2026-09-22.md
independent reference at chance (55 of 100)	docs/history/daily_notes/night_report_2026-09-14.md
letter bias (a) 88 %	round 54 in docs/history/preregistrations/prereg_round13_detector_push.md
CLIP gate about half right	Table 12 (49.3 %)
picture check validation	docs/history/preregistrations/freeze_picture_setup_2026-09-25.md
label re-checks, missing-sound checks	docs/history/preregistrations/prereg_round13_detector_push.md
test-set scoring (Section 9)	docs/history/preregistrations/prereg_round13_detector_push.md; docs/history/earlier_drafts/thesis/ch5_results.md

D Glossary of internal names

The development logs in `docs/history/` (release v1.2.0) and the scoring code use short codes. The main body of this report does not; this table maps each code to the plain name used in the report.

Table 10: Internal codes and their plain names.

Code	Plain name / meaning
<i>Systems and versions</i>	
D', SHIP8+MD3+WW5+SL	the final system (release v1.3.0, <code>config.use_shipped()</code>)
D	the final system with the scene check read as text
B, SHIP8+MD3	the version before the last three rules (witness rule, scene check, logit reading)
B0, B0r	the September baseline (<code>config.use_scored()</code>); B0r = B0 re-run in the same job as a candidate
B1	the September baseline with a BEATs self-veto (rejected)
SHIP1–SHIP8	successive accepted versions (Appendix E)
v1, v2, v3, v4	earlier versions of the system (August–September; Appendix E)
<i>Data</i>	
DEV-49, TEST-60	the older parts of the development and test sets
DEV2, TEST2	the newer parts (tagger clips)
merged DEV / TEST	the development set (71 clips) / the test set (88 clips)
the 415, held-out	the screening set (415 AudioSet-Strong clips)

Code	Plain name / meaning
slice B	30 AudioSet-Strong clips with a masked sound, reported separately
<i>Components</i>	
TO1+F7F8, TIER	listener rescue of weak FlexSED detections (Qwen alone for strong runs, both listeners for weak)
V4	a listener’s open list of heard sounds (“inventory”)
N2b	masked-weak veto
DR2	DASM rescue of new families
K-V4	a listener keep must also be named in an inventory
DV	DASM clip veto
BTP	band-twin pull (start a late picture at an earlier weak run)
CONT	continuation veto
FLAP	FineLAP veto on rescued sounds
K4AO, K4A-D	inventory keep (drop a picture neither listener lists; K4A-D: unless DASM hears it)
MD3	shortest detected sound $0.5 \rightarrow 0.3$ s
GRP, GROUP	same-family grouping by Qwen3-Omni
WW, WW5	witness rule; WW5 = witness rule with the scene check
SL, 60L	scene check read from the yes/no logits
DEPICT	depict-event drop
EXPECT	“expected sound” rules: name sounds expected off screen, then confirm (rejected)
<i>Process</i>	
Round n , R n	a numbered, pre-registered development round
amendment n	a numbered change to the September pre-registration (<code>prereg_v4.md</code>)

E Development history

This appendix is the record of how the final system was reached: how the data grew, the working rules, the earlier versions of the system, every accepted change, the main rejected ideas, the history of the pictures and of the evaluation, and a round-by-round table. Internal codes from the development logs appear here in parentheses so that each row can be found in `docs/history/` of release v1.2.0 (the full development record, Appendix C); Appendix D maps them to plain names.

E.1 How the data grew

Clip-level benchmark (August–September). 274 short clips (10–20s) were labelled by the author into four scenarios: unseen ambient (48 clips), mixed (25), seen ambient (132) and no ambient (69). Four sourcing strategies (keyword scraping, AudioSet strong segments, curated film scenes, UnAV-100 overlap moments) and an automatic filter all landed at 15–20% usable clips, which is why the set stopped at 274 rather than the planned 300. A test set of 100 clips (25 per scenario) was used by the early systems and the model judge (Appendix E.7).

Per-sound labels (22 September). After the judge-based evaluation was abandoned, clips were labelled per sound. The first export had 139 clips: 109 benchmark clips and 30 AudioSet-Strong clips with a consequential sound at least half under speech or music (“slice B”, always reported separately). The 109 were split by provenance into 49 development clips (“DEV-49”, clips already touched by earlier runs) and 60 test clips (“TEST-60”, labelled after the system was frozen). The September test table (Section E.9) is on TEST-60.

New clips (29 September – 1 October). 50 more clips were labelled with a stand-alone tool in three batches: 31 YouTube clips found by scene type and 19 AudioSet-Strong evaluation clips. They were split into 22 development (“DEV2”) and 28 test (“TEST2”) clips by a hash of the clip name or by balanced pairs, and merged on 30 Sept into the development set ($49 + 22 = 71$ clips) and the test set ($60 + 28 = 88$ clips) used in the body.

Table 11: Parts of the two evaluation sets.

Part	Clips	Sounds	Needed	Imp. 3	Clips w. needed
DEV-49	49	95	38	14	24
DEV2	22	59	20	3	14
TEST-60	60	94	43	11	28
TEST2	28	60	22	2	15

Needed-sound count. The development count moved during the last week: 55 at the first merged scoring, 58 after the visibility re-check (three sounds changed from seen to needed), 57 after a control round, and back to 58 after a correction. Every system was re-scored on each version; the test labels were never changed.

AudioSet sets. A calibration set of 280 AudioSet-Strong clips decided detector rounds 4–12 with a cost on timed AudioSet labels. Round 12 showed that under a corrected version of that cost the detector of the time was worse than showing nothing (1.650 vs 0.986 per clip), because AudioSet labels only a few sounds per clip; the set was retired as a decision test. The screening set of 415 clips was then used only for single audio rules, and a fresh set of 422 clips built for a last confirmation was never scored.

E.2 Working rules

From 13 Sept the following rules governed every change (`docs/history/plans/plan_robustness.md`, `docs/history/preregistrations/prereg_round13_detector_push.md`):

1. **Pre-register.** Before a run, write the idea, the exact rule, the data it is decided on and the pass bar, and commit it. Results are appended below the pre-registration, never edited into it.
2. **Decide on development data only.** The development set (or, for audio-only rules, the held-out 415) decides. The test set is scored after a change ships and is only reported.
3. **General rules only.** A rule must make sense for any video, run on one video at a time, and not name a clip or a single sound class (a rule that only ever fires on bells was rejected for this reason, Round 64).
4. **Compare with the shipped pipeline.** Every candidate is scored as a full-pipeline variant on the same clips as the current shipped system, not on a proxy; a component screen may stop an idea early but cannot ship it.
5. **Hits first.** Losing a few hits is acceptable only if many wrong pictures go (the fewer-pictures clause, Section 3.5).
6. **Consult and record.** Major choices were reviewed by panels of large-language-model reviewers (Claude, with assigned roles, two to five per panel, one to four rounds), whose reports are in `docs/history/review/` and `docs/panel_*.md`; every decision and its reason is in a dated log.

E.3 Phases

Table 12: Project phases. Scores in each row are in that phase’s own metric and are not comparable across rows.

Dates	System	Scored by	Outcome and why it changed
6 Aug	proposal: 7 stages (audio, VLM, ASR, BEATs/PANNs, reasoning, FLUX/SDXL, evaluation)	planned: VLM describes, LLM writes reference, LLM judges	pivot from an earlier speech-illustration project
7 Aug	v1: PANNs (bar 0.15) → Openverse image retrieval → side panel; no gate	demo clips	needed a “not seen” decision
7 Aug–7 Sept	CLIP zero-shot visibility gate; clip-level benchmark (274 clips)	clip-level gate accuracy	CLIP gate 49.3%: 71 of 103 seen clips still augmented – “object presence is not event visibility”
8–11 Sept	PANNs + OWLv2 + per-sound VLM gate (Qwen2.5-VL-7B, “name it, don’t confirm”); SDXL	100 clips × 3 systems, Mistral-7B judge 0–4, model-written reference	blind 2.94 > caption 2.68 > gated 2.41; retrieval beat SDXL by +0.17/+0.18; reference later found circular

Dates	System	Scored by	Outcome and why it changed
12–17 Sept	v3: BEATs (replaced PANNs after 11 of 14 complaints traced to its labels), hysteresis + occlusion onsets, 7B gate with 3 votes, FLUX.1-schnell	human-grounded judge reference, dev 174 / test 100	tie 3.17 vs 3.16; gate wins where nothing is due (+0.26), loses where a picture is due (−0.24)
18–21 Sept	v4: one state-of-the-art swap per stage; Qwen3.8-27B gate; FLAM and PretrainedSED detectors tried; rubric-enforced judge	judge with a cap on redundant pictures	PretrainedSED rejected three times; 13 silent bugs found (amendment 3), all earlier judge rows kept as history only
22–23 Sept	per-sound gold (139 clips); BEATs $\cup$ FlexSED (bar 0.8), FlexSED veto 0.3, PANNs veto 0.05	per-sound F1, then viewer cost (amendment 9)	F1 difference null (+0.028); wrong pictures and precision significant
24–27 Sept	monotone onsets, no length cap; final September test table (amendment 21); Qwen-Image frozen as picture setup	Holm-corrected table on TEST-60	“Holm wins, F1 null” (Section E.9)
27 Sept	detector rounds 1–3: cascade, PE-A-Frame, Qwen3-Omni as verifier	DEV-49 + AudioSet 280 / 415	none passed; detection declared closed, reopened next day
28 Sept	rounds 4–12 on AudioSet sets: EAT, Dasheng, DASM, SAM-Audio, 17 ideas, local contrast, DASM rescue, masker tests	cost on AudioSet labels	all failed on the 415; round 12 showed the AudioSet cost cannot track the task: harness retired
28–30 Sept	rounds 13–14: listener rescue (Qwen3-Omni, Audio Flamingo Next) + DASM vote; tagger clips added	DEV-49, then merged DEV	first gains in hits (TO1+F7F8)
30 Sept–1 Oct	ship chain SHIP1–SHIP8 on merged DEV/TEST	merged DEV, pass rules	DEV 18/51/3.690 $\rightarrow$ 28/21/2.282 (Table 13)
1–2 Oct	rounds 46–66: display rules, witness rule, scene check, many gate ideas	merged DEV, held-out 415	the final system: DEV 29/15/2.056; frozen on 2 Oct

E.4 The shipped chain

Table 13 is the backbone of the history: every change that entered the final system, in order, with its development and test numbers on the merged sets. Development numbers use the corrected gold

(58 needed sounds); test numbers are the reads made after each change shipped. Sources: docs/history/review/improvements_log_2026-09-30.md, docs/history/review/detector_final_2026-10-02.md, docs/history/preregistrations/prereg_round13_detector_push.md.

Table 13: Changes that shipped, from the old pipeline to the final system. Hits / wrong / cost; DEV hits out of 58, TEST out of 65. The p in the TEST column is one-sided, against the old pipeline B0r for SHIP5–SHIP8 and against B for D and the final system.

Step (name in logs)	What it adds	Merged DEV	Merged TEST
old pipeline (B0r)	September test-table system: BEATs $\cup$ FlexSED, FlexSED and PANNs vetoes, Qwen3.8 gate	18 / 51 / 3.690	21 / 40 / 2.909
TO1+F7F8 (R14)	listener rescue of weak FlexSED runs (tiered: Qwen alone above 0.6, both listeners below), one rescue per family, mirror veto with listener keep, DASM vote on rescues	26 / 45 / 3.070	22 / 38 / 2.818
+ N2b, DR2, K-V4 (R16–21)	masked-weak veto; DASM rescue of new families; a listener keep must also be named in an inventory	27 / 38 / 2.817	23 / 36 / 2.727
+ DV (R28)	drop a sound DASM never hears in the clip	27 / 31 / 2.620	23 / 34 / 2.682
+ BTP (R30)	start a late picture at an earlier weak run of the same sound	28 / 30 / 2.535	23 / 33 / 2.659 ($p = 0.033$)
+ CONT (R31)	drop a picture that is only a later piece of a sound already going on	28 / 25 / 2.394	24 / 32 / 2.591 ($p = 0.025$)
+ FineLAP veto (R31)	a 2025 frame-level model double-checks rescued sounds	28 / 24 / 2.366	24 / 31 / 2.568 ($p = 0.015$)
+ K4A-D = SHIP8 (R35)	drop a picture neither listener names, unless DASM clearly hears it	28 / 21 / 2.282	23 / 29 / 2.568 ($p = 0.017$)
+ merge gap 2.5 s	continuing sounds with short breaks no longer drew a second picture	28 / 20 / 2.254	23 / 28 / 2.545
+ grouping ≤ 8 s (R47)	Qwen3-Omni: same continuing sound or a new event?	28 / 18 / 2.197	22 / 26 / 2.545
+ shortest sound 0.3 s (R48)	short real sounds (a laugh burst) were thrown away	29 / 18 / 2.141	22 / 26 / 2.545
+ depict-event (R57)	drop a picture whose event is visibly happening	29 / 17 $\rightarrow$ 18 ^a / 2.141	22 / 26 / 2.545
= B (SHIP8+MD3)	base of the last rounds	29 / 18 / 2.141	22 / 26 / 2.545
+ witness rule + scene check (R53+R60) = D	weak sounds need DASM, both listeners, or one listener and a plausible scene	29 / 14 / 2.028 ^b	24 / 23 / 2.386
+ logit readout (R60L) = final system	scene check read as a probability, no cut-off answers	29 / 15 / 2.056	24 / 24 / 2.409 ($p = 0.034$ vs B)

^a Round 57 scored 29/17 with 26 listener answers missing from a cache; after the cache was filled (1 October) the same system scores 29/18/2.141, so its net development effect is zero. ^b D’s 2.028 contains one gain caused by a cut-off answer (“based on the”) that removed a wrong *Glass* picture; the logit readout of the final system cannot be cut off, so that picture returns (+1 wrong). The pass bar of Round 60L was set against B, not against D, for this reason.

The later per-step test p -values are uncorrected and were never used to accept a change.

What the table shows. From the old pipeline to the final system the development set gained 11 hits and lost 36 wrong pictures (cost 3.690 $\rightarrow$ 2.056). The test set gained 3 hits and lost 16 wrong pictures (2.909 $\rightarrow$ 2.409). About a third of the development gain in cost carried over to the test set; the test numbers are the better estimate of what a new video would see.

E.5 What was tried and rejected

Most ideas failed. Table 14 groups the main ones by the question they tried to answer, with the reason from their own pre-registered test. All were decided on the development set or the held-out 415.

Table 14: Main rejected ideas (selection). Hits / wrong / cost on merged DEV unless stated.

Idea (round)	What it did	Why it failed
<i>Hear more sounds</i>		
other detectors (Sept, rounds 1–8)	FLAM, PretrainedSED (5 backbones), AST, CED, SSLAM, EAT, Dasheng, PE-A-Frame, CLAP verifier, tagger ensembles	recall rose only with many more false spans; PretrainedSED lost three times; ensembles mis-scaled
source separation (Sept, R7)	HT-Demucs, SAM-Audio in front of BEATs	separated audio is out of the detectors’ training distribution: with HT-Demucs, recall under speech fell 9.5 % $\rightarrow$ 4.8 %; SAM-Audio failed its quality check (music never removed)
listener as verifier (R13)	ask Qwen3-Omni yes/no, multiple choice, open list on every weak candidate	finds 5–8 dropped real sounds but adds about 3 wrong pictures per hit
newer listeners (R31–34)	MOSS-Audio, SpotSound, Kimi, Step-Audio	3–10 wrong per extra hit; SpotSound answered yes to every query
EXPECT chain (R40–42b)	the scene VLM or a listener names sounds expected off screen; FineLAP times them; DASM confirms	best DEV 31/26/2.254 (+3 hits), but TEST 25/35/2.614 (worse); the gate cannot refuse a wrong off-screen name
short twins, band lists (R49, R56)	add weak or short detections both models half-hear	BANDLIST 0 hits, +6 wrong; TWIN-SHORT 27/36/2.761
tagger ensemble (R63)	calibrated EAT/SSLAM ensemble on BEATs windows	on the 415: 33 % fewer spans but recall 0.152 vs 0.197
<i>Remove wrong pictures</i>		
stricter witnesses (R53 variants)	require DASM or both listeners for every weak sound	27/13/2.113: –5 wrong but 2 real hits lost (fails the 3:1 clause)
named veto, relabel gate (R39–41)	drop or rename a picture when the gate or the listeners name a different sound	real hits lost for no cost gain
held-out family prior (R41)	drop families a detector gets wrong > 70 % of the time on the 415	every affected picture had been heard by a listener: no change
<i>Better on-screen decision</i>		
other gate signals (Sept, am. 14–20)	OWLv2 or SAM 3 vote, video input, synchrony, “any stretch” rule	each removed exactly as many needed sounds as wrong pictures it saved (break-even at $\beta = 2$)
SSL-SaN, SigLIP, BOX (R30–38)	audio-visual localiser; picture-vs-frame similarity; VLM draws a box on the source and checks the crop	each un-silenced real visible sounds; BOX-2 in the full pipeline 29/26/2.366

Idea	What it did	Why it failed
synchrony (R38)	Synchformer: does on-screen motion move with the sound?	as a veto, +3 needed kept but 5 visible calls lost; as an “add seen” vote, on the bar but changed no shipped picture
human-style chain (HUMAN, R50, R66)	name the source, find it, zoom, ask “is it making the sound now?”	the VLM names “nothing” on most stretches; NAME-ALL passed its gate screen but lost 3 real hits in the pipeline (26/12/2.141)
sign/event and logit gate (R54, R62)	ask for visible effects; read the gate as a logit margin	letter-position bias or “no” habit; best AUROC 0.647, below the 0.65 bar
look-alike reasoning (R61)	“no train here, so it is the washing machine”	works on the laundromat, but also explains away real sounds (a crying sound “explained” by a visible parrot)
bell rule (R64)	concealed actions (a bell in a tower) count as not visible	passed DEV (30/14/1.972) but fires only on bells: not general, not shipped
<i>Timing and grouping</i>		
change-point onsets (R33)	SEBB onset post-processing	7 hits lost
earliest onset (R38), repeat rules (R31, R35)	start at the first detector to hear a sound; require silence before a repeat	hits lost or no net gain
pause questions (R51, R55)	ask whether a 2-s gap is a pause or a new sound	neither the listener nor the detectors can tell a pause from a dropout
context ± 5 s (R59)	give the listener more context	it names what it <i>sees</i> , not what it hears

The main lesson. Two patterns recur in this table. First, the missing sounds can often be found – a listener asked about the whole clip names the hammer, the train, the bell – but every way of adding them brings two to four wrong pictures per hit, because nothing downstream can tell a right off-screen name from a wrong one. Second, every change to the gate moves along the same trade-off line: it silences more on-screen sounds only by silencing about as many needed ones. Section 8 measures both.

E.6 Pictures

The picture model changed four times: Openverse retrieval (v1), SDXL and SDXL-Turbo, FLUX.1-schnell (chosen over PixArt- Σ on twelve real prompts, 13 Sept), and Qwen-Image-2512 (frozen 25 Sept, shipped 28 Sept). On 9 Sept a retrieval-versus-SDXL ablation on the same 100 clips favoured retrieval by +0.17 (gated) and +0.18 (blind) judge points, but 55 % of retrieved images carried a non-commercial or no-derivatives licence and 595 uses drew on only 60 distinct images, so generation was kept. The decisive picture test was a blind *glance test* by the author on 25 Sept: 54 sounds from 50 never-annotated clips, each picture shown for 1.5 s at 384 px with no name, and the author typed what the sound was. FLUX was recognised 14/54 times; Qwen-Image with the same text 26/54 (+0.222 [+0.074, +0.370]); Qwen-Image with a richer subject text 32/54, but that text invented an object (a thud drawn as a door) and was rejected. One of the 26 Qwen-Image pictures was later found to send a false message (a telephone bell drawn as a desk bell), so the +22-point result is reported as “not clean”. A planned sealed confirmation sitting was cancelled on 28 Sept and replaced by the check-and-redraw loop of Section 5.5. A wording test the same day kept the hand-written subject table: 7 of 7 known cases right, against 3 of 7 for two generic VLM wording methods (docs/history/analyses/picture_sense_test_2026-09-28.md).

E.7 Earlier evaluation by model judge (superseded)

The proposal planned an automatic evaluation: a VLM describes the augmented video, an LLM writes a reference of what the viewer should learn, and an independent LLM scores the description against the reference on a 0–4 scale. It was built and run, and it failed in instructive ways:

1. **A reference written by the system’s own models is circular.** The first reference was written by the system’s VLM from the system’s detections; it rewarded whichever system repeated the detector (blind 2.94 > caption 2.68 > gated 2.41 on 9 Sept). A second judge of another family agreed with the first (quadratic $\kappa = 0.753$), which shows only that two judges can share a bias.
2. **A reference written by independent models cannot decide visibility.** A reference built by an audio LLM, CLAP, a different VLM and a different LLM agreed with the human “nothing is missing” label on 55 of 100 clips – chance. An audio-visual model asked with and without the soundtrack reached 49.5 % balanced accuracy (bar 67.7 %); a “gap-closing” score had AUROC 0.529 (bar 0.75).
3. **The rubric was asymmetric.** A withheld picture scored 0 where one was due, a redundant one about 3 where none was due, so the rubric punished staying silent – the very behaviour the gate exists for. A capped rubric (19 Sept) and a human-grounded reference fixed the direction but not the noise.
4. **Judges cannot tell a picture from a text tag.** The best judge (Gemma-4-31B seeing the pictures directly, passing all three trust checks against the author’s labels) scored pictures and text tags alike ($-0.04 [-0.22, +0.14]$), because it reads a tag as the label. Six automatic picture checkers also failed calibration against the author (for example $\kappa = 0.21$ to 0.45 against bars of 0.5).

A five-reviewer scoring panel on 19 Sept moved the primary score to per-sound matching against human labels (Section 3) and kept the judge as a secondary instrument. The answer to RQ4 is built on this history.

E.8 How the cost changed during development

Figure 10 (Section 7.1) shows the cost after each accepted change, from the first full pipeline to the final system. The development cost fell by 1.634 per clip and the test cost by 0.500; the gap is expected, because changes were chosen on the development set.

The last step deserves a note. The witness rule was first scored with its scene question read as text, which gave 2.028 (development) and 2.386 (test). That text answer was sometimes cut off after four words, and one cut-off answer happened to remove a wrong picture – an artefact of truncation, not a property of the rule. The final system reads the same question from the model’s yes/no scores, which cannot be cut off; this brings the wrong picture back and gives the reported numbers 2.056 and 2.409. The final system was accepted because it beats the version before the witness rule (2.141 on development), which was the pre-set bar; the text-read result is therefore not plotted as a separate step.

E.9 The September comparison of the on-screen check

Before the final detector existed, the on-screen check was compared with and without it on the 60 older test clips (same gate model and rule, the September detector); Table 15 gives two-sided p -values with Holm correction. The same pattern holds for the final system (Table 3): the check removes many wrong pictures at the price of a few hits, which F1, weighing both equally, does not reward.

Table 15: The September comparison of the on-screen check (60 older test clips).

Measure	With minus without	95 % interval	Holm p
Wrong pictures per clip	−0.517	[−0.800, −0.283]	< 0.001
Viewer cost per clip	−0.833		0.012
Precision	+0.137		0.010
Clips handled perfectly	+0.219		0.006
F1	+0.059	[−0.030, +0.144]	0.18 ^a

^a F1 was the primary endpoint; its p is uncorrected and not significant.

E.10 Comparisons between versions

The final development phase compared each accepted change with the version before it and, early on, with the September baseline (“B0r”). Paired clip bootstrap on the test set, 2,000 draws; d = mean per-clip cost difference; one-sided p (a two-sided value is about twice as large); see Table 16. These use 2,000 draws and one-sided p -values; the body uses 100,000 draws and two-sided p -values, so the two are not directly comparable. Version B scored 29 / 18 / 2.141 (development) and 22 /

Table 16: Comparisons between versions (test set, one-sided). Test-set reads made during development, uncorrected for multiplicity; none was used to accept a change.

Comparison	d	95 % interval	One-sided p
final system vs the version before its last three rules (B)	−0.136	[−0.295, 0.000]	0.034
same, scene check read as text (D) vs B	−0.159	[−0.341, −0.023]	0.0145
intermediate version (SHIP8) vs September baseline	−0.341	[−0.636, −0.045]	0.017
first listener stack (TO1+F7F8) vs September baseline, pre-registered	−0.091	[−0.250, +0.045]	0.132

26 / 2.545 (test); D scored 29 / 14 / 2.028 and 24 / 23 / 2.386. The four early test reads of 30 Sept were also corrected for multiplicity (Holm); none was significant after correction.

Repeat-merge gap. Re-reading the same saved pictures at different merge gaps moves wrong-picture counts but not the ranking (Table 17). The baseline gains more from a longer gap (many

Table 17: Repeat-merge gap: hits / wrong / cost at three gaps.

Gap (s)	SHIP8, dev.	Baseline, dev.	SHIP8, test	Baseline, test
1.5	28 / 23 / 2.338	18 / 57 / 3.859	23 / 30 / 2.591	21 / 44 / 3.000
2.0	28 / 21 / 2.282	18 / 51 / 3.690	23 / 29 / 2.568	21 / 40 / 2.909
3.0	28 / 18 / 2.197	17 / 43 / 3.521	22 / 27 / 2.568	20 / 33 / 2.795

of its pictures are repeats), so the final system (gap 2.5s) against the baseline (gap 2.0s) is not a like-for-like comparison.

E.11 Round-by-round record

Verdicts: shipped = accepted into the system; rejected = failed its development rule; stop = failed its screen on the screening set; hold = parked; no-op = changed no picture.

One row per round or main arm, in order. Results are hits / wrong / cost on merged DEV (55 or 58 needed, see Appendix E.1) unless marked; “415” = held-out AudioSet-Strong screen; “old DEV” = DEV-49. Full text, bars and line numbers: docs/history/preregistrations/prereg_v4.md (amendments), docs/prereg_round4..12*.md, docs/history/preregistrations/prereg_round13_detector_push.md (rounds 13–66). Counted from that file: rounds 13–28 ran 129 new full-pipeline variants (106 on DEV-49 alone), rounds 29–41 ran 21 more and 42 component

screens, rounds 42–57 ran 5 full variants and about 30 screens, rounds 58–66 four full variants and 15 screens.

Table 18: Round-by-round record (am. = amendment of the pre-registration).

Round	Idea	Result	Verdict
<i>September amendments (prereg_v4)</i>			
am. 5	per-sound gold, F1 headline	$\Delta F1 +0.028 [-0.022, +0.074]$; precision, false alarms significant	F1 null
am. 8	FlexSED union, bar 0.8	passes 3/3 rules	shipped
am. 9	viewer cost, $\beta = 2$	gated 3.36 vs blind 4.24; silence 2.79	reported metric
am. 10	FlexSED veto 0.3	4.12 $\rightarrow$ 3.59	shipped
am. 11–15	per-family bars; speech-removed view; family silence; any-stretch rule; OWLv2 vote	each worse or neutral	rejected
am. 16	PANNs veto 0.05	3.02 / 3.10	shipped
am. 17	tagger ensemble	CED mis-scaled	rejected
am. 18–20	visibility signals (video, SAM 3, synchrony)	all at break-even 2:1	rejected
am. 21	final September TEST table	Section E.9	reported
am. 22–25	cascade, PE-A-Frame, Qwen3-Omni verifier	17 hits, cost 4.82; AUROC 0.53; AUROC 0.661	rejected
<i>Rounds 4–12 (AudioSet harness, 28 Sept)</i>			
4	BEATs self-veto replaces PANNs veto	415 neutral; later worse on DEV and TEST (2.83 vs 2.63, p 0.043)	reverted 30 Sept
5	EAT, Dasheng	415 +0.087	fail
6	DASM as detector	415 +0.294 [+0.111, +0.511]	fail (kept as witness)
7, 7b	SAM-Audio separation	quality check failed	stop
8	17 ideas (I1–I10)	closest I7 -0.039 , p 0.061	fail
9	local contrast J2	415 pass, DEV 14 $\rightarrow$ 13 hits	not shipped
10	DASM rescue R0–R7	9 of 61 targets reachable	hold
11	masker tests	all worse	no pick
12	cost v2 on AudioSet	shipped 1.650 vs nothing 0.986	harness retired
<i>Rounds 13–28 (29–30 Sept)</i>			
13	listener rescue, many question forms; R13-1 twin-max	~ 3 wrong per extra hit; R13-1 TEST = B0r	R13-1 kept inside TO1
14	TO1+F7F8	old DEV 18/24/2.45 vs 2.78; merged 23/48/3.155; TEST 22/38/2.818 ($p = 0.13$)	shipped (SHIP1)
16	N2b masked-weak veto	23/44/3.042; TEST 22/35/2.750	shipped
19–20	DR2 DASM rescue	24/44/2.986	shipped (SHIP2)
21	K-V4 inventory keep	24/41/2.901	shipped (SHIP3)
22–27	N2c, stacks, K2 variants	N2c 22/38/2.930 lost a hit	rejected
28	DV DASM clip veto	24/34/2.704; TEST 23/34/2.682	shipped (SHIP4)
<i>Rounds 29–41 (30 Sept – 1 Oct)</i>			
29	F8-U, DV-L, DV-G; V4D; GA	+2 hits for +7 wrong; no-op; -5 hits; STOP; STOP	rejected
30	BTP band-twin pull	25/33/2.620; TEST 23/33/2.659	shipped (SHIP5)
30	K4AO; PIC-SIM; RT	held; nothing flips; no-op	held / rejected
31	CONT continuation veto	25/28/2.479; TEST 24/32/2.591	shipped (SHIP6)
31	FineLAP veto 0.329	25/27/2.451; TEST 24/31/2.568	shipped (SHIP7)
31	RPT-S, PMC, N2c, DVG, PTC, SUBJ, OM, SPOT, H2	screens or arms without gain	rejected
32	FineLAP in DASM’s seat; DV2	+11 wrong; Thunder lost	rejected
33	SPOT-V, change-point onsets, K4A-4E, SSL-SaN gate	all “Yes”; -7 hits; no-op; STOP	rejected
34	MOSS-Audio as listener	+4/ -9 needed, +63 other	rejected
35	DBX, DBR	premise wrong; cost not lower	rejected
re-check	visibility re-check (gold)	3/14 flips; control 0/20	gold change
35	K4A-D (K4AO + DASM keep)	28/58, 21/2.282; TEST 23/29/2.568 ($p = 0.017$)	shipped (SHIP8)
36–37	DASM impact peaks + VLM naming; BOX gate	+2 cross; -7 seen silenced	rejected
38	CF, BOX-2, GBTP, SHIP8+BOX2, SYNC, SYNC-2	best BOX-2 arm 29/26/2.366	rejected
39	DETACHED-ADD, CONTRAST, MAKER-VIS, RELABEL-GATE	+7 wrong; +142 other; -9 hits; hit lost	rejected

Round	Idea	Result	Verdict
40–40e	EXPECT chain	A4: DEV 31/26/2.254, TEST 25/35/2.614	not shipped
41	AGREE, NAMED-VETO, PRIOR415	below bar; hits lost; no change	rejected
<i>Rounds 42–57 (1 Oct)</i>			
42	held-out check of EXPECT-A4 audio chain	74/92 = 0.80 precise vs 0.38–0.39 raw	diagnostic
42b	EXPECT-A5 (7 vouched families)	DEV 30/21/2.169; TEST 23/31/2.614	not shipped
43–43c, 50	gate gold-set variants (HUMAN, HUMAN-2, HUMAN-ADD)	one sound short of the bar at best	rejected
44	AVNAME (eyes + ears name the sound)	rename 15/34; drop 27/17/2.225, TEST no change	rejected
45	AGREE-EARS (second listener family)	DEV 31/24/2.197; 415 0.833 < 0.85; TEST 2.659	rejected
46, 46b	gold scope (blind rating of wrong pictures)	≈1 in 6 is a real unlisted sound	no gold edit
–	merge gap 2.5 s	28/20/2.254	shipped
47	GRP-A grouping by Qwen3-Omni, ≤8 s	28/18/2.197	shipped
48	MD3 shortest span 0.3 s	29/18/2.141	shipped (= B)
49	BANDLIST	0 hits, +6 wrong	rejected
51, 52b, 55	pause rules, kin-keep, frame-pause	415 STOP	rejected
52	SK7 strong keep	29/22/2.254	rejected
53, 53d	WEAK-WITNESS alone	27/13/2.113 (2 hits lost); 27/17/2.225	fail alone
53b, 53c	witness bars from the 415	outside allowed range	stop
54, 54b	SIGN (visible effect)	0 flips; “no” on 9/10	rejected
56	TWIN-SHORT	27/36/2.761	rejected
57	DEPICT-EVENT	29/17 (net 0 after cache fill)	shipped
<i>Rounds 58–66 (1–2 Oct)</i>			
58, 58b	tight listener cut; FlexSED witness	415 precision 0.400, 0.297	stop
59	context ±5 s	415 0.333	stop
60	SCENE-MARGIN + witness (= D)	29/14/2.028; TEST 24/23/2.386 ($p = 0.0145$)	shipped
61b–d	look-alike reasoning (Qwen, Gemma)	hits lost in every variant	fail
62, 62b	logit gate; PRIOR	AUROC 0.647 / 0.644 < 0.65	stop
60L	scene check by logit margin (= the final system)	29/15/2.056; TEST 24/24/2.409 ($p = 0.034$)	shipped, frozen
63	tagger ensemble	415 recall 0.152 vs 0.197	stop
64	concealed action {Bell}	30/14/1.972 (one clip)	not general, not shipped
65	perceptual-onset return	415 0.115	stop
66	NAME-ALL crop margin	gate screen passed; DEV 26/12/2.141 (3 hits lost)	fail